

数据库原理综合习题答案

1.1 名词解释

(1) DB：即数据库 (Database),是统一管理的相关数据的集合。 DB 能为各种用户共享, 具有最小冗余度, 数据间联系密切, 而又有较高的数据独立性。

(2) DBMS：即数据库管理系统 (Database Management System), 是位于用户与操作系统之间的一层数据管理软件, 为用户或应用程序提供访问 DB 的方法, 包括 DB 的建立、查询、更新及各种数据控制。 DBMS 总是基于某种数据模型, 可以分为层次型、网状型、关系型、面向对象型 DBMS。

(3) DBS：即数据库系统 (Database System),是实现有组织地、动态地存储大量关联数据, 方便多用户访问的计算机软件、 硬件和数据资源组成的系统, 即采用了数据库技术的计算机系统。

(4) 1：1 联系：如果实体集 E1 中的每个实体最多只能和实体集 E2 中的一个实体有联系, 反之亦然, 那么实体集 E1 对 E2 的联系称为“一对一联系”, 记为“1：1”。

(5) 1：N 联系：如果实体集 E1 中每个实体与实体集 E2 中任意个 (零个或多个) 实体有联系, 而 E2 中每个实体至多和 E1 中的一个实体有联系, 那么 E1 对 E2 的联系是“一对多联系”, 记为“1：N”。

(6) M：N 联系：如果实体集 E1 中每个实体与实体集 E2 中任意个 (零个或多个) 实体有联系, 反之亦然, 那么 E1 对 E2 的联系是“多对多联系”, 记为“M：N”。

(7) 数据模型：模型是对现实世界的抽象。在数据库技术中, 表示实体类型及实体类型间联系的模型称为“数据模型”。它可分为两种类型：概念数据模型和结构数据模型。

(6) 概念数据模型：是独门于计算机系统的模型, 完全不涉及信息在系统中的表示, 只是用来描述某个特定组织所关心的信息结构。

(9) 结构数据模型：是直接面向数据库的逻辑结构, 是现实世界的第二层抽象。这类模型涉及到计算机系统和数据库管理系统, 所以称为“结构数据模型”。结构数据模型应包含：数据结构、数据操作、数据完整性约束三部分。它主要有：层次、网状、关系三种模型。

(10) 层次模型：用树型结构表示实体类型及实体间联系的数据模型。

(11) 网状模型：用有向图结构表示实体类型及实体间联系的数据模型。

(12) 关系模型：是目前最流行的数据库模型。其主要特征是用二维表格结构表达实体集, 用外键表示实体间联系。关系模型是由若干个关系模式组成的集合。

(13) 概念模式：是数据库中全部数据的整体逻辑结构的描述。它由若干个概念记录类型组成。概念模式不仅要描述概念记录类型, 还要描述记录间的联系、操作、数据的完整性、安全性等要求。

(14) 外模式：是用户与数据库系统的接口, 是用户用到的那部分数据的描述。

(15) 内模式：是数据库在物理存储方面的描述, 定义所有的内部记录类型、索引和文件的组成方式, 以及数据控制方面的细节。

(16) 模式 / 内模式映象：这个映象存在于概念级和内部级之间, 用于定义概念模式和内模式间的对应性, 即概念记录和内部记录间的对应性。此映象一般在内模式中描述。

(17) 外模式 / 模式映象：这人映象存在于外部级和概念级之间, 用于定义外模式和概念模式间的对应性, 即外部记录和内部记录间的对应性。此映象都是在外模式中描述。

(18) 数据独立性：在数据库技术中, 数据独立性是指应用程序和数据之间相互独立, 不受影响。数据独立性分成物理数据独立性和逻辑数据独立性两级。

(19) 物理数据独立性：如果数据库的内模式要进行修改, 即数据库的存储设备和存储方法有所变化, 那么模式 / 内模式映象也要进行相应的修改, 使概念模式尽可能保持不变。也就是对模式的修改尽量不影响概念模式。

(20) 逻辑数据独立性：如果数据库的概念模式要进行修改（如增加记录类型或增加数据项），那么外模式 / 模式映象也要进行相应的修改，使外模式尽可能保持不变。也就是对概念模式的修改尽量不影响外模式和应用程序。

(21) 宿主语言：在数据库技术中，编写应用程序的语言仍然是一些高级程序设计语言，这些语言称为宿主语言（host language), 简称主语言。

(22) DDL：数据定义语言（Data Definition Language), 用于定义数据库的三级结构，包括外模式、概念模式、内模式及其相互之间的映象，定义数据的完整性、安全控制等约束。

(23) DML：数据操纵语言（Data Manipulation Language), 由 DBMS 提供，用于让用户或程序员使用，实现对数据库中数据的操作。DML 分成交互型 DML 和嵌入式 DML 两类。依据语言的级别，DML 又可分成过程性 DML 和非过程性 DML 两种。

(24) 交互型 DML：如果 DML 自成系统，可在终端上直接对数据库进行操作，这种 DML 称为交互型 DML。

(25) 嵌入式 DML：如果 DML 嵌入在主语言中使用，此时主语言是经过扩充能处理 DML 语句的语言，这种 DML 称为嵌入式 DML。

(26) 过程性 DML：用户编程时，不仅需要指出“做什么”（需要什么样的数据），还需要指出“怎么做”（怎么获得数据）。层状、网状的 DML 属于过程性语言。

(27) 非过程性 DML：用户编程时，只需要指出“做什么”，不需要指出“怎么做”。

Notice: 以上关于 DML 的各个概念单独出现时，首先要解释 DML 的含义。

(28) DD :数据字典（Data Dictionary), 数据库系统中存放三级结构定义的数据库称为数据字典。对数据库的操作都要通过访问 DD 才能实现。

(29) DD 系统：管理 DD 的实用程序称为“DD 系统”。

1.2 文件系统阶段的数据管理有什么缺陷？试举例说明。

文件系统有三个缺陷：

（1）数据冗余性（redundancy）。由于文件之间缺乏联系，造成每个应用程序都有对应的文件，有可能同样的数据在多个文件中重复存储。

（2）数据不一致性（inconsistency）。这往往是由数据冗余造成的，在进行更新操作时，稍不谨慎，就可能使同样的数据在不同的文件中不一样。

（3）数据联系弱（poor data relationship）。这是由文件之间相互独立，缺乏联系造成的。

1.3 数据库阶段的数据管理有什么特点？

(1) 采用复杂的数据模型表示数据结构

(2) 有较高的数据独立性（数据结构分成用户的逻辑结构、整体逻辑结构和物理结构三级）

(3) 数据库系统为用户提供方便的用户接口，可以使用查询语言、终端命令或程序方式操作数据，也可以用程序方式操作数据库。

(4) 系统提供了四个方面的数据控制功能：数据库的恢复、并发控制、数据完整性和数据安全性，以保证数据库中数据是安全的、正确的和可靠的。

(5) 对数据的操作不一定以记录为单位，还可以数据项为单位，增加了系统的灵活性。

1.4 你怎样理解实体、属性、记录、字段这些概念的类型和值的差别？试举例说明。

实体（entity）：是指客观存在可以相互区别的事物。实体可以是具体的对象，如：一个学生，一辆汽车等；也可以是抽象的事件，如：一次借书、一场足球赛等。

属性（attribute): 实体有很多特性，每一个特性称为属性。每个属性有一个值域，其类型可以是整数型、实数型、字符串型。比如，学生（实体）有学号、姓名、年龄、性别等属性，

相应值域为字符、字符串、整数和字符串型。

字段（field）: 标记实体属性的命名单位称为字段或数据项。它是可以命名的最小信息单位，所以又称为数据元素或初等项。字段的命名往往和属性名相同，比如，学生有学号、姓名、年龄、性别等字段。

记录（record）: 字段的有序集合称为记录。一般用一个记录描述一个实体，所以记录又可以定义为能完整地描述一个实体的字段集。如：一个学生记录，由有序的字段集（学号、姓名、年龄、性别等）组成。

1.5 逻辑记录与物理记录，逻辑文件与物理文件有什么联系和区别？

联系：

- (1) 逻辑记录与物理记录都是记录，是字段的有序集合；
- (2) 逻辑文件与物理文件都是文件，是同一类记录的汇集。

区别：

- (1) 逻辑记录与逻辑文件是逻辑数据描述，物理记录与物理文件是物理数据描述。
- (2) 物理数据描述是指数据在存储设备上的存储方式，物理记录、物理文件（还有物理联系、物理结构等术语），都是用来描述实际存储设备上的数据。
- (3) 逻辑数据描述是指程序员或用户用以操作的数据形式，是抽象的概念化数据。逻辑记录、逻辑文件（还有逻辑联系、逻辑结构等术语），都是用户观点的数据描述。

1.6 为某百货公司设计一个 ER 模型。

百货管辖若干个连锁商店，每家商店经营若干商品，每家商店有若干职工，但每个职工只能服务于一家商店。实体类型“商店”的属性有：商店编号，店名，店址，店经理。实体类型“商品”的属性有：商品编号，商品名，单价，产地。实体类型“职工”的属性有：职工编号，职工名，性别，工资。在联系中应反映出职工参加某商店工作的开始时间，商店销售商品的月销售量。试画出反映商店、商品、职工实体类型及联系类型的 ER 图，并将其转换成关系模式集。

实体：商店（商店编号，店名，店址，店经理）

商品（商品编号，商品名，单价，产地）

职工（职工编号，职工名，性别，工资）

联系：SC（商店—商品之间 1：N 的联系，联系属性为“职工参加商店工作的开始时间”。SE（商店—职工之间 1：N 的联系），联系属性为“月销售量”。

关系模式集：商店模式（商店编号，店名，店址，店经理）

商品模式（商品编号，商品名，单价，产地，商店编号，月销售量）

职工模式（职工编号，职工名，性别，工资，商店编号，开始时间）

1.7 试述 ER 模型、层次模型、网状模型、关系模型和面向对象模型的主要特点。

ER 模型的主要特点：

- (1) 优点：接近于人的思维，容易理解；与计算机无关，用户容易接受。
- (2) 缺点：只能说明实体间语义的联系，不能进一步说明详细的数据结构。

层次模型的特点：

- (1) 优点：记录之间的联系通过指针实现，查询效率较高。

(2) 缺点：只能表示 1:N 联系，实现 M:N 结构较复杂；由于层次顺序的严格和复杂，引起数据的查询和更新操作也很复杂。

网状模型的特点：

(1) 优点：记录之间联系通过指针实现，M:N 联系也容易实现（每个 M:N 联系可拆成两个 1:N 联系），查询效率较高。

(2) 缺点：编写应用程序比较复杂，程序员必须熟悉数据库的逻辑结构。

关系模型的特点：

用关键码而不是用指针导航数据，表格简单，用户易懂，编程时并不涉及存储结构、访问技术等细节。

1.8 试述概念模式在数据库结构中的重要地位。

概念模式是数据库中全部数据的整体逻辑结构的描述。它由若干个概念记录类型组成。概念模式不仅要描述概念记录类型，还要描述记录间的联系、操作、数据的完整性、安全性等要求。数据按外模式的描述提供给用户，按内模式的描述存储在磁盘中，而概念模式提供了连接这两级的相对稳定的中间观点，并使得两级的任何一级的改变都不受另一级的牵制。

1.9 数据独立性与数据联系这两个概念有什么区别？

数据独立性是指应用程序与数据之间相互独立，不受影响。

数据联系是指同一记录内部各字段间的联系，以及记录之间的联系。

1.10 试述 DBMS 在用户访问数据库过程中所起的作用。

用户对数据库进行操作，DBMS 把操作从应用程序带到外部级、概念级、再导向内部级，进而操作存储器中的数据。

（结合 P22 “用户访问数据的过程”来理解）

1.11 试述 DBMS 的主要功能。

DBMS 的主要功能有：

- (1) 数据库的定义功能
- (2) 数据库的操纵功能
- (3) 数据库的保护功能
- (4) 数据库的存储管理
- (5) 数据库的维护功能
- (6) 数据字典

1.12 试叙 DBMS 对数据库的保护功能。

DBMS 对数据库的保护主要通过四个方面实现：

- (1) 数据库的恢复。
- (2) 数据库的并发控制。
- (3) 数据库的完整性控制。
- (4) 数据库的安全性控制。

1.13 试叙 DBMS 对数据库的维护功能。

DBMS 中有一些程序提供给数据库管理员运行数据库系统时使用，这些程序起着数据库维护的功能。

主要有四个实用程序：

- (1) 数据装载程序 (loading)
- (2) 备份程序 (backup)
- (3) 文件重组织程序
- (4) 性能监控程序

1.14 从模块结构看，DBMS 由哪些部分组成？

从模块结构看，DBMS 由两大部分组成：查询处理器和存储管理器

(1) 查询处理器有四个主要成分：DDL 编译器，DML 编译器，嵌入型 DML 的预编译器，查询运行核心程序

(2) 存储管理器有四个主要成分：授权和完整性管理器，事务管理器，文件管理器，缓冲区管理器

(以上几题具体可参照书上 p20-21)

1.15 DBS 由哪几个部分组成？

DBS 由四部分组成：数据库、硬件、软件、数据库管理员。

1.16 什么样的人 DBA？DBA 应具有什么素质？DBA 的职责是什么？

DBA 是控制数据整体结构的人，负责 DBS 的正常运行。DBA 可以是一个人，在大型系统中也可以是由几个人组成的小组。DBA 承担创建、监控和维护整个数据库结构的责任。

DBA 应具有下列素质：

- (1) 熟悉企业全部数据的性质和用途；
- (2) 对用户的需求有充分的了解；
- (3) 对系统的性能非常熟悉。

DBA 的主要职责有五点：

- (1) 概念模式定义
- (2) 内模式定义
- (3) 根据要求修改数据库的概念模式和内模式
- (4) 对数据库访问的授权
- (5) 完整性约束的说明

1.17 试对 DBS 的全局结构作详细解释。

参照教材 p24-25。

1.18 使用 DBS 的用户有哪几类？

使用 DBS 的用户有四类：

- 1) DBA
- 2) 专业用户
- 3) 应用程序员
- 4) 最终用户

1.19 DBMS 的查询处理器有哪些功能？

DBMS 的查询处理器可分成四个成分：

- 1) DML 编译器
- 2) 嵌入型 DML 的预编译器
- 3) DDL 编译器
- 4) 查询运行核心程序

(各成分功能参照 P24)

1.20 DBMS 的存储处理器有哪些功能？

DBMS 的存储处理器提供了应用程序访问数据库中数据的界面，可分成四个成分：

- 1) 授权和完整性管理器
- 2) 事务管理器
- 3) 文件管理器
- 4) 缓冲区管理器

(各成分功能参照 P25)

1.21 磁盘存储器中有哪四类主要的数据结构？

数据文件：存储了数据库中的数据；

数据字典（DD）：存储三级结构的描述；

索引文件：为提高查询速度而设置的逻辑排序手段；

统计数据组织：存储 DBS 运行时统计分析数据。

(1) 关系模型：用二维表格结构表示实体集，外键表示实体间联系的数据模型称为关系模型。

(2) 关系模式：关系模式实际上就是记录类型。它的定义包括：模式名，属性名，值域名以及模式的主键。关系模式不涉及到物理存储方面的描述，仅仅是对数据特性的描述。

(3) 关系实例：元组的集合称为关系和实例，一个关系即一张二维表格。

(4) 属性：实体的一个特征。在关系模型中，字段称为属性。

(5) 域：在关系中，每一个属性都有一个取值范围，称为属性的值域，简称域。

(6) 元组：在关系中，记录称为元组。元组对应表中的一行；表示一个实体。

(7) 超键：在关系中能唯一标识元组的属性集称为关系模式的超键。

(8) 候选键：不含有多余属性的超键称为候选键。

(9) 主键：用户选作元组标识的一个候选键为主键。（单独出现，要先解释“候选键”）

(10) 外键：某个关系的主键相应的属性在另一关系中出现，此时该主键在就是另一关系的外键，如有两个关系 S 和 SC 其中 S# 是关系 S 的主键，相应的属性 S# 在关系 SC 中也出现，此时 S# 就是关系 SC 的外键。

(11) 实体完整性规则：这条规则要求关系中元组在组成主键的属性上不能有空值。如果出现空值，那么主键值就起不了唯一标识元组的作用。

(12) 参照完整性规则：这条规则要求“不引用不存在的实体”。其形式定义如下：如果属性

集 K 是关系模式 R_1 的主键, K 也是关系模式 R_2 的外键, 那么 R_2 的关系中, K 的取值只允许有两种可能, 或者为空值, 或者等于 R_1 关系中某个主键值。这条规则在使用时有三点应注意: 1) 外键和相应的主键可以不同名, 只要定义在相同值域上即可。 2) R_1 和 R_2 也可以是同一个关系模式, 表示了属性之间的联系。 3) 外键值是否允许空应视具体问题而定。

(13) 过程性语言: 在编程时必须给出获得结果的操作步骤, 即“干什么”和“怎么干”。如 Pascal 和 C 语言等。

(14) 非过程性语言: 编程时只须指出需要什么信息, 不必给出具体的操作步骤。各种关系查询语言均属于非过程性语言。

(15) 无限关系: 当一个关系中存在无穷多个元组时, 此关系为无限关系。如元组表达式 $\{t | \neg R(t)\}$ 表示所有不在关系 R 中的元组的集合, 这是一个无限关系。

(16) 无穷验证: 在验证公式时需对无穷多个元组进行验证就是无穷验证。如验证公式 $(u)(P(u))$ 的真假时需对所有的元组 u 进行验证, 这是一个无穷验证的问题。

2.2 为什么关系中的元组没有先后顺序?

因为关系是一个元组的集合, 而元组在集合中的顺序无关紧要。因此不考虑元组间的顺序, 即没有行序。

2.3 为什么关系中不允许有重复元组?

因为关系是一个元组的集合, 而集合中的元素不允许重复出现, 因此在关系模型中对关系作了限制, 关系中的元组不能重复, 可以用键来标识唯一的元组。

2.4 关系与普通的表格、文件有什么区别?

关系是一种规范化了的二维表格, 在关系模型中, 对关系作了下列规范性限制:

- 1) 关系中每一个属性值都是不可分解的。
- 2) 关系中不允许出现相同的元组 (没有重复元组)。
- 3) 由于关系是一个集合, 因此不考虑元组间的顺序, 即没有行序。
- 4) 元组中, 属性在理论上也是无序的, 但在使用时按习惯考虑列的顺序。

2.5 笛卡尔积、等值联接、自然联接三者之间有什么区别?

笛卡尔积对两个关系 R 和 S 进行乘操作, 产生的关系中元组个数为两个关系中元组个数之积。

等值联接则是在笛卡尔积的结果上再进行选择操作, 从关系 R 和 S 的笛卡尔积中选择对应属性值相等的元组;

自然连接则是在等值联接 (以所有公共属性值相等为条件) 的基础上再行投影操作, 并去掉重复的公共属性列。当两个关系没有公共属性时, 自然连接就转化我笛卡尔积。

2.8 如果 R 是二元关系, 那么下列元组表达式的结果是什么?

$\{t(u)(R(t) \wedge R(u) \wedge (t[1] \neq u[1] \vee t[2] \neq u[2]))\}$

这个表达式的意思是: 从关系 R 中选择元组, 该元组满足: 第 1 分量值或第 2 分量值至少有一个不等于其他某元组。由于 R 是二元关系, 只有两个分量, 由于没有重复元组, 上述条

件显然满足。所以，这个表达式结果就是关系 R 。

2.9 假设 R 和 S 分别是三元和二元关系，试把表达式 $\pi_{1,5}(\sigma_{2=4 \vee 3=4}(R \times S))$ 转换成等价的：

(1) 汉语查询句子； (2) 元组表达式； (3) 域表达式。

(1) 汉语表达式：

从 $R \times S$ 关系中选择满足下列条件的元组：

第 2 分量（ R 中第 2 分量）与第 4 分量（ S 中第 1 分量）值相等，或第 3 分量（ R 中第 3 分量）与第 4 分量（ S 中第 1 分量）值相等；并取第 1 列与第 5 列组成的新关系。

(2) 元组表达式： $\{t(u)(v)(R(u) \wedge S(v) \wedge (u[2]=v[1] \vee u[3]=v[1]) \wedge t[1]=u[1] \wedge t[2]=v[2])\}$

(3) 域表达式： $\{xv|(y)(z)(u)(R(xyz) \wedge S(uv) \wedge (y=u \vee z=u))\}$

2.10 假设 R 和 S 都是二元关系，试把元组表达式 $\{t|R(t) \wedge (u)(S(u) \wedge u[1] \neq t[2])\}$ 转换成等价的： (1) 汉语查询句子； (2) 域表达式； (3) 关系代数表达式。

(1) 汉语表达式：选择 R 关系中元组第 2 分量值不等于 S 关系中某元组第 1 分量值的元组。

(2) 域表达式： $\{xy|(u)(v)(R(xy) \wedge S(uv) \wedge (u \neq y))\}$

(3) 关系代数表达式： $\pi_{1,2}(\sigma_{2 \neq 3}(R \times S))$

2.11 试把域表达式 $\{ab|R(ab) \wedge R(ba)\}$ 转换成等价的：

(1) 汉语查询句子； (2) 关系代数表达式； (3) 元组表达式。

(1) 汉语查询句子：选择 R 中元组第 1 分量值与第 2 分量值互换后仍存在于 R 中的元组。

(2) 关系代数表达式： $\pi_{1,2}(\sigma_{1=2}(R \times R))$ ；

(3) 元组表达式： $\{t|(u)(R(t) \wedge R(u) \wedge t[1]=u[2] \wedge t[2]=u[1])\}$

2.12 设有两个关系 $R(A, B, C)$ 和 $S(D, E, F)$ ，试把下列关系代数表达式转换成等价的元组表达式：

(1) $\pi_A(R)$ ；(2) $\sigma_{B='17'}(R)$ ；(3) $R \times S$ ；(4) $\pi_{A,F}(\sigma_{C=D}(R \times S))$

(1) $\{t|(u)(R(u) \wedge t[1]=u[1])\}$

(2) $\{t|R(t) \wedge t[2]='17'\}$

(3) $\{t|(u)(v)(R(u) \wedge S(v) \wedge t[1]=u[1] \wedge t[2]=u[2] \wedge t[3]=u[3] \wedge t[4]=v[1] \wedge t[5]=v[2] \wedge t[6]=v[3])\}$

(4) $\{t|(u)(v)((R(u) \wedge S(v) \wedge u[3]=v[1] \wedge t[1]=u[1] \wedge t[2]=v[3])\}$

2.13 设有三个关系：

$S(S\#, SNAME, AGE, SEX)$

$SC(S\#, C\#, GRADE)$

$C(C\#, CNAME, TEACHER)$

试用关系代数表达式表示下列查询语句。（见下一题）

2.14 试用元组表达式表示上题中各个查询语句。

(1) 检索 LIU 老师所授课程的课程号、课程名。

$\pi_{C\#, CNAME}(\sigma_{TEACHER='LIU'}(C))$

$\{t|(u)(C(u) \wedge C[3]='LIU' \wedge t[1]=u[1] \wedge t[2]=u[2])\}$

(2)检索年龄大于 23 岁的男学生的学号与姓名。

$\pi S\#,SNAME(\sigma AGE>'23' \wedge SEX=男'(S))$
 $\{t(u)(S(u) \wedge u[3]>'23' \wedge u[4]='男' \wedge t[1]=u[1] \wedge t[2]=u[2])\}$

(3)检索学号为 S3 学生所学课程的课程名与任课教师名。

$\pi CNAME,TEACHER(\sigma S\#='S3'(SC))$
 $\{t(u)(v)(SC(u) \wedge C(v) \wedge u[1]='S3' \wedge v[1]=u[2] \wedge t[1]=v[2] \wedge t[2]=v[3])\}$

(4)检索至少选修 LIU 老师所授课程中一门课程的女学生的姓名。

$\pi SNAME(\sigma SEX=女' \wedge TEACHER='LIU'(SSCC))$
 $\{t(u)(v)(w)(S(u) \wedge SC(v) \wedge C(w) \wedge u[4]='女' \wedge v[1]=u[1] \wedge v[2]=w[1] \wedge w[3]='LIU' \wedge t[1]=u[2])\}$

(5)检索 WANG 同学不学的课程号。

$\pi C\#(C)-\pi C\#(\sigma SNAME='WANG'(SSC))$
或者,
 $\pi C\#(SC)-\pi C\#(\sigma SNAME='WANG'(SSC))$ (全部课程号减去 WANG 同学所学的课程号)
 $\{t(u)(v)(C(u) \wedge SC(v) \wedge (u[1]=v[2] \Rightarrow (w)(s(w) \wedge w[1]=v[1] \wedge w[2] \neq 'wang')) \wedge t[1]=u[1])\}$
(从 C 中选择满足条件的元组: SC 中的所有元组 ,如果学号与 C 中所选元组相同的话, 其在 S 中对应的姓名肯定不是 'wang'。)

Notice:" $p1 \Rightarrow p2$ " 的含义是: 如果 $p1$ 为真, 则 $p2$ 为真。

(6)检索至少选修两门课程的学生学号。

$\pi S\#(\sigma 1 \neq 2(SC \times SC))$
SC 自乘之后, 再选择 (同一个学号中两个课程号不同的元组), 投影。
 $\{t(u)(v)(SC(u) \wedge SC(v) \wedge u[1]=v[1] \wedge u[2] \neq v[2]) \wedge t[1]=u[1]\}$

(7)检索全部学生都选修的课程的课程号与课程名。

$\pi C\#,CNAME(C(\pi S\#,C\#(SC) \div \pi S\#(S)))$ (涉及到全部值时, 应用除法, “除数”是“全部”)
 $\{t(u)(v)(w)(S(u) \wedge SC(v) \wedge C(w) \wedge u[1]=v[1] \wedge v[2]=w[1] \wedge t[1]=v[1] \wedge t[2]=v[2])\}$

(8)检索选修课程包含 LIU 老师所授课程的学生学号。

$\pi S\#(\sigma TEACHER='LIU'(SC))$
 $\{t(u)(v)(SC(u) \wedge C(v) \wedge u[2]=v[1] \wedge v[3]='LIU' \wedge t[1]=u[1])\}$

如果 LIU 老师有多门课程, 则选修课程包含 LIU 老师所授全部课程的学生学号为:

$\pi S\#,C\#(SC) \div \pi C\#(\sigma TEACHER='LIU'(C))$

2.15 在教学数据库 S、SC、C 中, 用户有一查询语句: 检索女同学选修课程的课程名和任课教师名。 (1)试写出该查询的关系代数表达式; (2)试写出查询优化的关系代数表达式。

(1) $\pi CNAME,TEACHER(\sigma SEX=女'(SSCC))$

(2) 优化为: $\pi CNAME, TEACHER(\sigma C\#(\pi S\#,C\#(SC) \div \pi S\#(\sigma SEX=女'(S))))$

(基本思路: 尽量提前做选择操作; 在每个操作后, 应做个投影操作, 去掉不用的属性值。)

2.16 在 2.15 题中,

- (1)画出该查询初始的关系代数表达式的语法树。
- (2)使用 2.4.4 节的优化算法，对语法树进行优化，并画出优化后的语法树。

该查询初始的关系代数表达式的语法树
优化后的语法树

2.17 为什么要对关系代数表达式进行优化？

在关系代数运算中，各个运算所费时间和空间是不一样的。如何安排若干关系的运算操作步骤，直接影响到整个操作所需要的时间和空间。对关系代数表达式进行优化，可以提高系统的操作效率，达到执行过程即省时间又省空间的目的。

3.1 名词解释

(1)SQL 模式：SQL 模式是表和授权的静态定义。一个 SQL 模式定义为基本表的集合。一个由模式名和模式拥有者的用户名或账号来确定，并包含模式中每一个元素（基本表、视图、索引等)的定义。

(2)SQL 数据库：SQL(Structured Query Language)，即‘结构式查询语言’，采用英语单词表示和结构式的语法规则。一个 SQL 数据库是表的汇集，它用一个或多个 SQL 模式定义。

(3)基本表：在 SQL 中，把传统的关系模型中的关系模式称为基本表 (Base Table)。基本表是实际存储在数据库中的表，对应一个关系。

(4)存储文件：在 SQL 中，把传统的关系模型中的存储模式称为存储文件 (Stored File)。每个存储文件与外部存储器上一个物理文件对应。

(5)视图：在 SQL 中，把传统的关系模型中的子模式称为视图 (View)，视图是从若干基本表和（或）其他视图构造出来的表。

(6)行：在 SQL 中，把传统的关系模型中的元组称为行 (row)。

(7)列：在 SQL 中，把传统的关系模型中的属性称为列 (column)。

(8)实表：基本表被称为“实表”，它是实际存放在数据库中的表。

(9)虚表：视图被称为“虚表”，创建一个视图时，只把视图的定义存储在数据词典中，而不存储视图所对应的数据。

(10)相关子查询：在嵌套查询中出现的符合以下特征的子查询：子查询中查询条件依赖于外层查询中的某个值，所以子查询的处理不只一次，要反复求值，以供外层查询使用。

(11)联接查询：查询时先对表进行笛卡尔积操作，然后再做等值联接、选择、投影等操作。
联接查询的效率比嵌套查询低。

(12)交互式 SQL：在终端交互方式下使用的 SQL语言称为交互式 SQL。

(13)嵌入式 SQL：嵌入在高级语言的程序中使用的 SQL语言称为嵌入式 SQL。

(14)共享变量：SQL和宿主语言的接口。共享变量有宿主语言程序定义，再用 SQL的 DECLARE 语句说明，SQL语句就可引用这些变量传递数据库信息。

(15)游标：游标是与某一查询结果相联系的符号名，用于把集合操作转换成单记录处理方式。

(16)卷游标：为了克服游标在推进时不能返回的不便，SQL2提供了卷游标技术。卷游标在推进时不但能沿查询结果中元组顺序从头到尾一行行推进，也能一行行返回。

3.2 对于教学数据库的三个基本表

学生 S(S#,SNAME,AGE,SEX)

学习 SC(S#,C#,GRADE)

课程 C(C#,CNAME,TEACHER)

试用 SQL的查询语句表达下列查询：

(1) 检索 LIU 老师所授课程的课程号和课程名。

```
SELECT C# CNAME
FROM C
WHERE TEACHER='LIU '
```

(2) 检索年龄大于 23 岁的男学生的学号和姓名。

```
SELECT S# SNAME
FROM S
WHERE (AGE>23) AND (SEX='M ')
```

(3) 检索至少选修 LIU 老师所授课程中一门课程的女学生姓名。

```
SELECT SNAME
FROM S
WHERE SEX='F ' AND S# IN
(
  SELECT S#
  FROM SC
  WHERE C# IN
  (
    SELECT C#
```

FR

OM C

```
WHERE TEACHER='LIU ')
```

NOTICE: 有多种写法，比如联接查询写法：

```
SELECT SNAME
FROM S,SC,C
WHERE SEX='F ' AND SC.S#=S.S#
AND SC.C#=C.C#
```

AND TEACHER='LIU'

但上一种写法更好一些。

(4) 检索 WANG 同学不学的课程的课程号。

```
SELECT C#
FROM C
WHERE C# NOT IN
  (SELECT C#
   FROM SC
   WHERE S# IN
     (SELECT S#
      FROM S
      WHERE SNAME='WANG'))
```

(5) 检索至少选修两门课程的学生学号。

```
SELECT DISTINCT X.SNO
FROM SC X,SC Y
WHERE X.SNO=Y.SNO AND X.CNO<>Y.CNO
```

Notice: 对表 SC 进行自连接, X, Y 是 SC 的两个别名。

(6) 检索全部学生都选修的课程的课程号与课程名。

```
SELECT C#,CNAME
FROM C
WHERE NOT EXISTS
  (SELECT *
   FROM S
   WHERE S# NOT IN
     (SELECT *
      FROM SC
      WHERE SC.C#=C.C#))
```

要从语义上分解：（1）选择课程的课程号与课程名，不存在不选这门课的同学。

其中，“不选这门课的同学”可以表示为：

```
SELECT *
```

```
FROM S
```

```
WHERE S# NOT IN
```

```
(SELECT *
```

```
FROM SC
```


WHERE SC.C#=C.C#)

或者

SELECT *

FROM S

WHERE NOT EXISTS

(SELECT *

FROM SC

WHERE S.S#=C.S# AND

SC.C#=C.C#)

(7) 检索选修课程包含 LIU 老师所授课的学生学号。

SELECT DISTINCT S#

FROM SC

WHERE C# IN

(SELECT C#

FROM C

WHERE TEACHER='LIU'))

3.3 设有两个基本表 R (A, B, C) 和 S (D, E, F)，试用 SQL 查询语句表达下列关系代数表达式：

(1) $\pi A(R)$ (2) $\sigma B='17'(R)$ (3) $R \times S$ (4) $\pi A,F(\sigma C=D(R \times S))$

(1) SELECT A FROM R

(2) SELECT * FROM R WHERE B='17'

(3) SELECT A,B,C,D,E,F FROM R,S

(4) SELECT A,F FROM R,S WHERE R.C=S.D

3.4 3.4 设有两个基本表 R (A, B, C) 和 S (A, B, C) 试用 SQL 查询语句表达下列关系代数表达式：

(1) $R \cup S$ (2) $R \cap S$ (3) $R - S$ (4) $\pi_{A,B}(R) \pi_{B,C}(S)$

```
(1)SELECT A,B,C
    FROM R
    UNION
    SELECT A,B,C
    FROM S
```

```
(2)SELECT A,B,C
    FROM R
    INTERSECT
    SELECT A,B,C
    FROM S
```

```
(3)SELECT A,B,C
    FROM R
    WHERE NOT EXISTS
        (SELECT A,B,C
         FROM S
         WHERE R.A=S.A AND R.B=S.B AND R.C=S.C)
```

```
(4)SELECT R.A,R.B,S.C
    FROM R,S
```

WHE

RE R.B=S.B

3.5 试叙述 SQL语言的关系代数特点和元组演算特点。
(P61-62)

3.6 试用 SQL查询语句表达下列对教学数据库中三个基本表 S、SC、C的查询：

(1) 统计有学生选修的课程门数。

```
SELECT COUNT(DISTINCT C#) FROM SC
```

(2) 求选修 C4 课程的学生的平均年龄。

```
SELECT AVG(AGE)
FROM S
WHERE S# IN
    (SELECT S#
     FROM SC
     WHERE C#='C4')
```

或者,

```
SELECT AVG(AGE)
FROM S, SC
```

```
WHERE S.S#=SC.S# AND C#='004'
```

- (3) 求 LIU 老师所授课程的每门课程的学生平均成绩。

```
SELECT CNAME,AVG(GRADE)
FROM SC ,C
WHERE SC.C#=C.C# AND TEACHER='LIU'
GROUP BY C#
```

- (4) 统计每门课程的学生选修人数（超过 10 人的课程才统计）。要求输出课程号和选修人数，查询结果按人数降序排列，若人数相同，按课程号升序排列。

```
SELECT DISTINCT C#,COUNT(S#)
FROM SC
GROUP BY C#
HAVING COUNT(S#)>10
ORDER BY 2 DESC, C# ASC
```

- (5) 检索学号比 WANG 同学大，而年龄比他小的学生姓名。

```
SELECT X.SNAME
FROM S AS X, S AS Y
WHERE Y.SNAME='WANG' AND X.S#>Y.S# AND X.AGE<Y.AGE
```

- (6) 检索姓名以 WANG 打头的所有学生的姓名和年龄。

```
SELECT SNAME,AGE
FROM S
WHERE SNAME LIKE 'WANG%'
```

- (7) 在 SC 中检索成绩为空值的学生学号和课程号。

```
SELECT S#,C#
FROM SC
WHERE GRADE IS NULL
```

- (8) 求年龄大于女同学平均年龄的男学生姓名和年龄。

```
SELECT SNAME,AGE
FROM S AS X
WHERE X.SEX='男' AND X.AGE>(SELECT AVG(AGE) FROM S AS Y WHERE Y.SEX='女')
```

- (9) 求年龄大于所有女同学年龄的男学生姓名和年龄。

```
SELECT SNAME,AGE
FROM S AS X
WHERE X.SEX='男' AND X.AGE>ALL (SELECT AGE FROM S AS Y WHERE Y.SEX='女')
```

3.7 试用 SQL 更新语句表达对教学数据库中三个基本表 S、SC、C 的各个更新操作：

- (1) 往基本表 S 中插入一个学生元组（ ' S9'，' WU'，18）。

```
INSERT INTO S(S#,SNAME,AGE) VALUES('59','WU',18)
```

(2) 在基本表 S 中检索每一门课程成绩都大于等于 80 分的学生学号、姓名和性别，并把检索到的值送往另一个已存在的基本表 STUDENT(S#, SNAME, SEX)。

```
INSERT INTO STUDENT(S#,SNAME,SEX)
SELECT S#,SNAME,SEX
FROM S WHERE NOT EXISTS
(SELECT * FROM SC WHERE
GRADE<80 AND S.S#=SC.S#)
```

(3) 在基本表 SC 中删除尚无成绩的选课元组。

```
DELETE FROM SC
WHERE GRADE IS NULL
```

(4) 把 WANG 同学的学习选课和成绩全部删去。

DELE

```
TE FROM SC
WHERE S# IN
(SELECT S#
FROM S
WHERE SNAME='WANG')
```

(5) 把选修 MATHS 课不及格的成绩全改为空值。

```
UPDATE SC
SET GRADE=NULL
WHERE GRADE<60 AND C# IN
(SELECT C#
FROM C
WHERE CNAME='MATHS')
```

(6) 把低于总平均成绩的女同学成绩提高 5%。

```
UPDATE SC
SET GRADE=GRADE*1.05
WHERE GRADE<(SELECT AVG(GRADE)FROM SC) AND S# IN (SELECT S# FROM S WHERE
SEX='F')
```

(7) 在基本表 SC 中修改 C4 课程的成绩，若成绩小于等于 75 分时提高 5%，若成绩大于 75 分时提高 4%（用两个 UPDATE 语句实现）。

```
UPDATE SC
SET GRADE=GRADE*1.05
WHERE C#='C4' AND GRADE<=75
UPDATE SC
SET GRADE=GRADE*1.04
WHERE C#='C4' AND GRADE>75
```


3.8 在第 1 章例 1.4 中提到“仓库管理”关系模型有五个关系模式：

零件 PART (P#, PNAME, COLOR, WEIGHT)

项目 PROJECT (J#, JNAME, DATE)

供应商 SUPPLIER (S#, SNAME, SADDR)

供应 P__P (J#, P#, TOTAL)

采购 P__S (P#, S#, QUANTITY)

(1) 试用 SQL DDL 语句定义上述五个基本表，并说明主键和外键。

```
CREATE TABLE PART
```

```
(P# CHAR(4) NOT NULL,PNAME CHAR(12) NOT NULL,
```

```
COLOR CHAR(10),WEIGHT REAL,
```

```
PRIMARY KEY(P#))
```

```
CREATE TABLE PROJECT
```

```
(J# CHAR(4) NOT NULL,JNAME CHAR(12) NOT NULL,
```

```
DATE DATE,
```

```
PRIMARY KEY(J#))
```

```
CREATE TABLE SUPPLIER
```

```
(S# CHAR(4) NOT NULL,SNAME CHAR(12),SADDR VARCHAR(20),
```

```
PRIMARY KEY(S#))
```

```
CREATE TABLE P__P
```

```
(J# CHAR(4),P# CHAR(4),TOTAL INTEGER,
```

```
PRIMARY KEY(J#,P#),
```

```
FOREIGN KEY(J#) REFERENCE PROJECT(J#),
```

```
FOREIGN KEY(P#) REFERENCE PART(P#))
```

```
CREATE TABLE P__S
```

```
(P# CHAR(4),S# CHAR(4),QUANTITY INTEGER,
```

```
PRIMARY KEY(P#,S#),
```

```
FOREIGN KEY(P#) REFERENCE PART(P#),
```

```
FOREIGN KEY(S#) REFERENCE SUPPLIER(S#))
```

(2) 试将 PROJECT P__P、PART三个基本表的自然联接定义为一个视图 VIEW1, PART P__S、SUPPLIER三个基本表的自然联接定义为一个视图 VIEW2。

```
CREATE VIEW VIEW1(J#,JNAME,DATE,P#,PNAME,COLOR,WEIGHT,TOTAL)
```

```
AS SELECT PROJECT.J#,JNAME,DATE,PART.P#,PNAME,COLOR,WEIGHT,TOTAL
```

```
FROM PROJECT,PART,P__P
```

```
WHERE PART.P#=P__P.P# AND P__P.J#=PROJECT.J#
```

```
CREATE VIEW VIEW2(P#,PNAME,COLOR,WEIGHT,S#,SNAME,SADDR,QUANTITY)
```

```
AS SELECT
```

PART.P#,PNAME,COLOR,WEIGHT,SUPPLIER.S#,SNAME,SADDR,QUANTITY

FROM PART,P_S,SUPPLIER
WHERE PART.P#=P_S.P# AND P_S.S#=SUPPLIER.S#

(3) 试在上述两个视图的基础上进行数据查询：

1) 检索上海的供应商所供应的零件的编号和名字。

SELECT P#,PNAME FROM VIEW2 WHERE SADDR='SHANGHAI'

2) 检索项目 J4 所用零件的供应商编号和名字。

SELECT S#,SNAME FROM VIEW2 WHERE P# IN(SELECT P# FROM VIEW1 WHERE J#='J4')

3.9 对于教学数据库中基本表 SC 已建立下列视图：

CREATE VIEW S__GRADE(S#, C__NUM, AVG__GRADE)
AS SELECT S#, COUNT(C#), AVG(GRADE)
FROM SC
GROUP BY S#

试判断下列查询和更新是否允许执行。若允许，写出转换到基本表 SC 上的相应操作。

(1) SELECT *

FROM S__GRADE

允许

SELECT S#,COUNT(C#),AVG(GRADE) FROM SC GROUP BY S#

(2) SELECT S#, C__NUM

FROM S__GRADE

WHERE AVG__GRADE> 80

允许

SELECT S#,COUNT(C#) FROM SC WHERE AVG(GRADE)>80

(3) SELECT S#, AVG__GRADE

FROM S__GRADE

WHERE C__NUM>(SELECT C__NUM

FROM S__GRADE

WHERE S# = ' S4')

允许

SELECT S#,AVG(GRADE)

FROM SC AS X

WHERE COUNT(X.C#)>(SELECT COUNT(Y.C#) FROM SC AS Y WHERE Y.S#='S4')

GROUP BY S#

(4) UPDATE S__GRADE

SET C__NUM = C__NUM + 1

WHERE S# = ' S4'

不允许

```
(5) DELETE FROM S__GRADE
      WHERE C__NUM > 4
```

不允许

3.10 预处理方式对于嵌入式 SQL的实现有什么重要意义？

预处理方式是先用预处理程序对源程序进行扫描，识别出 SQL 语句，并处理成宿主语言的函数调用形式；然后再用宿主语言的编译程序把源程序编译成目标程序。这样，不用扩充宿主语言的编译程序，就能处理 SQL语句。

3.11 在宿主语言的程序中使用 SQL语句有哪些规定？

在宿主语言的程序中使用 SQL语句有以下规定：

- (1) 在程序中要区分 SQL 语句与宿主语言语句
- (2) 允许嵌入的 SQL语句引用宿主语言的程序变量（称为共享变量），但有两项规定：
 - 1) 引用时，这些变量前必须加“：”作为前缀标识，以示与数据库中变量有区别。
 - 2) 这些变量由宿主语言的程序定义，并用 SQL的 DECLARE语句说明。
- (3) SQL 的集合处理方式与宿主语言单记录处理方式之间要协调。需要采用游标机制，把集合操作转换成单记录处理方式。

3.12 SQL 的集合处理方式与宿主语言单记录处理方式之间如何协调？

由于 SQL 语句处理的是记录集合，而宿主语言语句一次只能处理一个记录，因此需要用游标(cousor) 机制，把集合操作转换成单记录处理方式。

2.13 嵌入式 SQL语句何时不必涉及到游标？何时必须涉及到游标？

(1)INSERT、DELETE、UPDATE语句，查询结果肯定是单元组时的 SELECT语句，都可直接嵌入在主程序中使用，不必涉及到游标。

(2)当 SELECT语句查询结果是多个元组时，此时宿主语言程序无法使用，一定要用游标机制把多个元组一次一个地传送给宿主语言处理。

4.1 名词解释

(1) 函数依赖：FD(function dependency)，设有关系模式 $R(U)$ ， X, Y 是 U 的子集， r 是 R 的任一具体关系，如果对 r 的任意两个元组 t_1, t_2 ，由 $t_1[X]=t_2[X]$ 导致 $t_1[Y]=t_2[Y]$ ，则称 X 函数决定 Y ，或 Y 函数依赖于 X ，记为 $X \rightarrow Y$ 。 $X \rightarrow Y$ 为模式 R 的一个函数依赖。

(2) 函数依赖的逻辑蕴涵：设 F 是关系模式 R 的一个函数依赖集， X, Y 是 R 的属性子集，如果从 F 中的函数依赖能够推出 $X \rightarrow Y$ ，则称 F 逻辑蕴涵 $X \rightarrow Y$ ，记为 $F \models X \rightarrow Y$ 。

(3) 部分函数依赖：即局部依赖，对于一个函数依赖 $W \rightarrow A$ ，如果存在 XW (X 包含于 W) 有 $X \rightarrow A$ 成立，那么称 $W \rightarrow A$ 是局部依赖，否则称 $W \rightarrow A$ 为完全依赖。

(4) 完全函数依赖：见上。

(5) 传递依赖：在关系模式中，如果 $Y \rightarrow X$ ， $X \rightarrow A$ ，且 XY (X 不决定 Y)， AX (A 不属于 X)，那么称 $Y \rightarrow A$ 是传递依赖。

(6) 函数依赖集 F 的闭包 F^+ ：被逻辑蕴涵的函数依赖的全体构成的集合，称为 F 的闭包

(closure), 记为 F^+ 。

(7) 1NF：第一范式。如果关系模式 R 的所有属性的值域中每一个值都是不可再分解的值，则称 R 是属于第一范式模式。如果某个数据库模式都是第一范式的，则称该数据库模式属于第一范式的数据库模式。第一范式的模式要求属性值不可再分裂成更小部分，即属性项不能是属性组合和组属性组成。

(8) 2NF：第二范式。如果关系模式 R 为第一范式，并且 R 中每一个非主属性完全函数依赖于 R 的某个候选键，则称是第二范式模式；如果某个数据库模式中每个关系模式都是第二范式的，则称该数据库模式属于第二范式的数据库模式。（注：如果 A 是关系模式 R 的候选键的一个属性，则称 A 是 R 的主属性，否则称 A 是 R 的非主属性。）

(9) 3NF：第三范式。如果关系模式 R 是第二范式，且每个非主属性都不传递依赖于 R 的候选键，则称 R 是第三范式的模式。如果某个数据库模式中的每个关系模式都是第三范式，则称为 3NF 的数据库模式。

(10) BCNF：BC 范式。如果关系模式 R 是第一范式，且每个属性都不传递依赖于 R 的候选键，那么称 R 是 BCNF 的模式。

(11) 4NF：第四范式。设 R 是一个关系模式， D 是 R 上的多值依赖集合。如果 D 中成立非平凡多值依赖 $X \twoheadrightarrow Y$ 时， X 必是 R 的超键，那么称 R 是第四范式的模式。

(12) 推理规则的正确性和完备性：正确性是指，如果 $X \rightarrow Y$ 是从推理规则推出的，那么 $X \rightarrow Y$ 在 F^+ 中。完备性是指，不能从 F 使用推理规则导出的函数依赖不在 F^+ 中。

(13) 依赖集的覆盖和等价：关系模式 $R(U)$ 上的两个函数依赖集 F 和 G ，如果满足 $F^+ = G^+$ ，则称 F 和 G 是等价的。如果 F 和 G 等价，则可称 F 覆盖 G 或 G 覆盖 F 。

(14) 最小依赖集：如果函数集合 F 满足以下三个条件：(1) F 中每个函数依赖的右部都是单属性；(2) F 中的任一函数依赖 $X \rightarrow A$ ，其 $F - \{X \rightarrow A\}$ 与 F 是不等价的；(3) F 中的任一函数依赖 $X \rightarrow A$ ， Z 为 X 的子集， $(F - \{X \rightarrow A\}) \cup \{Z \rightarrow A\}$ 与 F 不等价。则称 F 为最小函数依赖集合，记为 F_{min} 。

(15) 无损联接：设 R 是一关系模式，分解成关系模式 $\rho = \{R_1, R_2, \dots, R_k\}$ ， F 是 R 上的一个函数依赖集。如果对 R 中满足 F 的每一个关系 r 都有 $r = \pi R_1(r) \pi R_2(r) \dots \pi R_k(r)$ 则称这个分解相对于 F 是“无损联接分解”。

(16) 保持依赖集：所谓保持依赖就是指关系模式的函数依赖集在分解后仍在数据库中保持不变，即关系模式 R 到 $\rho = \{R_1, R_2, \dots, R_k\}$ 的分解，使函数依赖集 F 被 F 这些 R_i 上的投影蕴涵。

(17) 多值依赖：设 $R(U)$ 是属性集 U 上的一个关系模式， X, Y, Z 是 U 的子集，并且 $Z = U - X - Y$ 用 x, y, z 分别代表属性集 X, Y, Z 的值，只要 r 是 R 的关系， r 中存在元组 (x, y_1, z_1) 和 (x, y_2, z_2) 时，就也存在元组 (x, y_1, z_2) 和 (x, y_2, z_1) ，那么称多值依赖 (MultiValued Dependency MVD) $X \twoheadrightarrow Y$ 在关系模式 R 中成立。

4.2 关系模式 R 有 n 个属性，在模式 R 上可能成立的函数依赖有多少个？其中平凡的函数依赖有多少个？非平凡的函数依赖有多少个？

(要考虑所有可能的情况，数学排列组合问题。对于数据库本身而言，本题没多大意义)

所有属性相互依赖时，函数依赖最多。

· 平凡的函数依赖：对于函数依赖 $X \rightarrow Y$ ，如果 $Y \subseteq X$ 那么称 $X \rightarrow Y$ 是一个“平凡的函数依赖”。

4.3 建立关于系、学生、班级、社团等信息的一个关系数据库，一个系有若干个专业，每个

专业每年只招一个班， 每个班有若干个学生， 一个系的学生住在同一宿舍区， 每个学生可以参加若干个社团， 每个社团有若干学生。

描述学生的属性有：学号、姓名、出生年月、系名、班级号、宿舍区。

描述班级的属性有：班级号、专业名、系名、人数、入校年份。

描述系的属性有：系名、系号、系办公地点、人数。

描述社团的属性有：社团名、成立年份、地点、人数、学生参加某社团的年份。

请给出关系模式， 写出每个关系模式的最小函数依赖集， 指出是否存在传递函数依赖， 对于函数依赖左部是多属性的情况， 讨论函数依赖是完全函数依赖还是部分函数依赖。 指出各关系的候选键、外部键， 有没有全键存在 ？

各关系模式如下：

学生 (学号 ,姓名 ,出生年月 ,系名 ,班级号 ,宿舍区)

班级 (班级号 ,专业名 ,系名 ,人数 ,入校年份)

系 (系名 ,系号 ,系办公地点 ,人数)

社团 (社团名 ,成立年份 ,地点 ,人数)

加入社团（社团名， 学号， 学生参加社团的年份）

学生 (学号 ,姓名 ,出生年月 ,系名 ,班级号 ,宿舍区)

● “ 学生 ” 关系的最小函数依赖集为：

$F_{min}=\{\text{学号} \rightarrow \text{姓名}, \text{学号} \rightarrow \text{班级号}, \text{学号} \rightarrow \text{出生年月}, \text{学号} \rightarrow \text{系名}, \text{系名} \rightarrow \text{宿舍区}\}$

●以上关系模式中存在传递函数依赖， 如 :学号→系名， 系名→宿舍区

●候选键是学号 ,外部键是班级号 ,系名。

notice: 在关系模式中， 如果 $Y \rightarrow X, X \rightarrow A$, 且 XY (X 不决定 Y)， A 不属于 X ,那么称 $Y \rightarrow A$ 是传递依赖。

班级 (班级号 ,专业名 ,系名 ,人数 ,入校年份)

● “ 班级 ” 关系的最小函数依赖集为：

$F_{min}=\{(\text{系名}, \text{专业名}) \rightarrow \text{班级号}, \text{班级号} \rightarrow \text{人数}, \text{班级号} \rightarrow \text{入校年份}, \text{班级号} \rightarrow \text{系名}, \text{班级号} \rightarrow \text{专业名}\}$

(假设没有相同的系， 不同系中专业名可以相同)

●以上关系模式中不存在传递函数依赖。

● “ $(\text{系名}, \text{专业名}) \rightarrow \text{班级号}$ ” 是完全函数依赖。

●候选键是 $(\text{系名}, \text{专业名})$,班级号， 外部键是系名。

系 (系名 ,系号 ,系办公地点 ,人数)

● “ 系 ” 关系的最小函数依赖集为： $F_{min}=\{\text{系号} \rightarrow \text{系名}, \text{系名} \rightarrow \text{系办公地点}, \text{系名} \rightarrow \text{人数}, \text{系名} \rightarrow \text{系号}\}$

●以上关系模式中不存在传递函数依赖

●候选键是系名 ,系号

社团 (社团名 ,成立年份 ,地点 ,人数)

- “ 社团 ” 关系的最小函数依赖集为： $F_{min}=\{\text{社团名} \rightarrow \text{成立年份}, \text{社团名} \rightarrow \text{地点}, \text{社团名} \rightarrow \text{人数}\}$
- 以上关系模式中不存在传递函数依赖。
- 候选键是社团名

加入社团 (社团名, 学号, 学生参加社团的年份)

- “ 加入社团 ” 关系的最小函数依赖集为： $F_{min}=\{(\text{社团名}, \text{学号}) \rightarrow \text{学生参加社团的年份}\}$
- “ (社团名, 学号) $\rightarrow$ 学生参加社团的年份 ” 是完全函数依赖。
- 以上关系模式中不存在传递函数依赖。
- 候选键是 (社团名, 学号) 。

4.4 对函数依赖 $X \rightarrow Y$ 的定义加以扩充, X 和 Y 可以为空属性集, 用 ϕ 表示, 那么 $X \rightarrow \phi$, $\phi \rightarrow Y$, $\phi \rightarrow \phi$ 的含义是什么 ?

根据函数依赖的定义, 以上三个表达式的含义为:

(1) 一个关系模式 $R(U)$ 中, X, Y 是 U 的子集, r 是 R 的任一具体关系, 如果对 r 的任意两个元组 t_1, t_2 , 由 $t_1[X]=t_2[X]$ 必有 $t_1[\phi]=t_2[\phi]$ 。即 $X \rightarrow \phi$ 表示空属性函数依赖于 X 。这是任何关系中都存在的。

(2) $\phi \rightarrow Y$ 表示 Y 函数依赖于空属性。由此可知该关系中所有元组中 Y 属性的值均相同。

(3) $\phi \rightarrow \phi$ 表示空属性函数依赖于空属性。这也是任何关系中都存在的。

4.5 已知关系模式 $R(ABC)$ $F=\{A \rightarrow C, B \rightarrow C\}$, 求 F^+ 。

可以直接通过自反律、增广律、传递律加以推广:

$F^+=\{\phi \rightarrow \phi, A \rightarrow \phi, B \rightarrow \phi, C \rightarrow \phi, A \rightarrow C, B \rightarrow C, AB \rightarrow \phi, AB \rightarrow A, AB \rightarrow B, AB \rightarrow C, AB \rightarrow BC, AB \rightarrow AB, AB \rightarrow ABC, BC \rightarrow \phi, BC \rightarrow C, BC \rightarrow B, BC \rightarrow BC, AC \rightarrow \phi, AC \rightarrow C, AC \rightarrow A, AC \rightarrow AC, ABC \rightarrow \phi, ABC \rightarrow A, ABC \rightarrow B, ABC \rightarrow C, ABC \rightarrow BC, ABC \rightarrow AB, ABC \rightarrow ABC\}$

4.6 试分析下列分解是否具有无损联接和保持函数依赖的特点:

(1) 设 $R(ABC)$ $F_1=\{A \rightarrow B\}$ 在 R 上成立, $\rho_1=\{AB, AC\}$ 。

首先, 检查是否具有无损联接特点:

第 1 种解法 -- 算法 4.2:

A
B
C

AB
a1
a2
b13

AC
a1
b22
a3

A
B
C

a1
a2
b13

a1
a2
a3

- (1) 构造表
- (2)根据 $A \rightarrow B$ 进行处理

结果第二行全是 a 行，因此分解是无损联接分解。

第 2 种解法 :(定理 4.8)

设 $R1=AB$, $R2=AC$

$R1 \cap R2 = A$

$R2 - R1 = B$

$\therefore A \rightarrow B, \therefore$ 该分解是无损联接分解。

然后，检查分解是否保持函数依赖

$\pi R1 (F1) = \{A \rightarrow B, \text{以及按自反率推出的一些函数依赖} \}$

$\pi R_2(F_1) = \{\text{按自反率推出的一些函数依赖}\}$

F_1 被 $\pi R_1(F_1)$ 所蕴涵， $\therefore$ 所以该分解保持函数依赖。

(2) 设 $R(ABC)$ $F_2 = \{A \rightarrow C, B \rightarrow C\}$ 在 R 上成立， $\rho_2 = \{AB, AC\}$

首先，检查是否具有无损联接特点：

第 1 种解法（略）

第 2 种解法：(定理 4.8)

设 $R_1 = AB$, $R_2 = AC$

$R_1 \cap R_2 = A$

$R_2 - R_1 = C$

$\therefore A \rightarrow C, \therefore$ 该分解是无损联接分解。

然后，检查分解是否保持函数依赖

$\pi R_1(F_2) = \{\text{按自反率推出的一些函数依赖}\}$

$\pi R_2(F_2) = \{A \rightarrow C, \text{以及按自反率推出的一些函数依赖}\}$

$\therefore F_1$ 中的 $B \rightarrow C$ 没有被蕴涵，所以该分解没有保持函数依赖。

(3) 设 $R(ABC)$ $F_3 = \{A \rightarrow B\}$, 在 R 上成立， $\rho_3 = \{AB, BC\}$.

首先，检查是否具有无损联接特点：

第 1 种解法：

A

B

C

AB

a1

a2

b13

BC

b21

a2

a3

A

B

C

a1

a2

a3

a1

b22

a3

(1) 构造表

(2) 根据 $A \rightarrow B$ 进行处理

没有一行全是 a 行。因此这个分解不具有无损联接特性。

第 2 种解法 :(定理 4.8)

设 $R_1=AB$, $R_2=BC$

$R_1 \cap R_2 = B$

$R_2 - R_1 = C$, $R_1 - R_2 = A$

$\because B \rightarrow C, B \rightarrow A$ 不在 F_3 中 $\therefore$ 该分解不具有无损联接特性。

然后, 检查分解是否保持函数依赖

$\pi R_1(F_3) = \{A \rightarrow B, \text{以及按自反率推出的一些函数依赖}\}$

$\pi R_2(F_3) = \{\text{按自反率推出的一些函数依赖}\}$

F_1 被 $\pi R_1(F_3)$ 所蕴涵, 所以该分解保持函数依赖。

(4) 设 $R(ABC)$ $F_4 = \{A \rightarrow B, B \rightarrow C\}$ 在 R 上成立, $\rho_4 = \{AC, BC\}$

首先, 检查是否具有无损联接特点:

第 1 种解法 (略)

第 2 种解法 :(定理 4.8)

设 $R_1=AC$, $R_2=BC$

$R_1(AC) \cap R_2(BC) = C$

$R_2 - R_1 = B$, $R_1 - R_2 = A$

$\because C \rightarrow B, C \rightarrow A$ 不在 F_4 中 $\therefore$ 该分解不具有无损联接特性。

然后, 检查分解是否保持函数依赖

$\pi R_1(F_2) = \{\text{按自反率推出的一些函数依赖}\}$

$\pi R_2(F_2) = \{B \rightarrow C, \text{以及按自反率推出的一些函数依赖}\}$

$\therefore F_1$ 中的 $A \rightarrow B$ 没有被蕴涵, 所以该分解没有保持函数依赖。

4.7 设 $R=ABCD$, R 上的函数依赖集 $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow D, D \rightarrow C\}$, R 的一个分解 $\rho = \{AB, AC, AD\}$, 求: (1) F 在 ρ 的每个模式上的投影。 (2) ρ 相对于 F 是无损联接分解吗? (3) ρ 保持依赖吗?

(1)

$\pi_{AB}(F)=\{A\rightarrow B,$ 及按自反律所推导出的一些平凡函数依赖 $\}$
 $\pi_{AC}(F)=\{A\rightarrow C,$ 及按自反律所推导出的一些平凡函数依赖 $\}$
 $\pi_{AD}(F)=\{A\rightarrow D,$ 及按自反律所推导出的一些平凡函数依赖 $\}$
(2)

A
B
C
D

AB
a1
a2
b13
b14

AC
a1
b22
a3
b24

AD
a1
b32
b33
a4

A
B
C
D

a1
a2
a3
a4

a1
a2
a3
a4

a1
a2
a3
a4

- (1) 构造表
- (2)根据 $A \rightarrow B, B \rightarrow C, A \rightarrow D, D \rightarrow C$ 进行处理

每一行都是 a, ρ 相对于 F 是无损联接分解。

(3) $\pi_{AB}(F) \cup \pi_{AC}(F) \cup \pi_{AD}(F) = \{A \rightarrow B, A \rightarrow C, A \rightarrow D\}$,
没有满足 $B \rightarrow C, D \rightarrow C$ 函数依赖,
因此 ρ 相对于 F 的这个分解不保持函数依赖。

4.8 设 $R=ABCD$,R上的 $F=\{A \rightarrow C, D \rightarrow C, BD \rightarrow A\}$, 试证明 $\rho =\{AB,ACD,BCD\}$ 相对于 F 不是无损联接分解。
根据算法 4.2

A
B
C
D

AB
a1
a2
b13
b14

ACD
a1
b22
a3
a4

BCD

b31
a2
a3
a4

A
B
C
D

a1
a2
a3
b14

a1
b22
a3
a4

b31
a2
a3
a4

- (1) 构造表
- (2)根据 $A \rightarrow C, D \rightarrow C, BD \rightarrow A$ 进行处理

没有一行都是 a，所以， ρ 相对于 F 不是无损联接分解。

4.9 设 $R=ABCD$,R上的 $F=\{A \rightarrow B, B \rightarrow C, D \rightarrow B\}$, 把 R 分解成 BCNF 模式集。

- (1)若首先把 R 分解成 $\{ACD,BD\}$ 试求 F 在这两个模式上的投影。
- (2)ACD 和 BD 是 BCNF 吗?如果不是，请进一步分解。

- (1) $\pi_{ACD}(F)=\{A \rightarrow C\}$
 $\pi_{BD}(F)=\{D \rightarrow B\}$

(2)因为根据 BCNF 的定义， 要求关系模式是第一范式， 且每个属性都不传递依赖于 R 的候选键。 BCD 中（ A， D）为候选键，可是（ A,D) $\rightarrow$ A, $A \rightarrow$ C,所以它不是 BCNF 模式。

它可进一步分解为： {AC,DC}此时 AC, DC均为 BCNF模式。

BD 是 BCNF, 因为 R2(BD)是第一范式， 且每个属性都不传递依赖于 D(候选键), 所以它是 BCNF 模式。

4.10 设 $R=ABCD, \rho =\{AB,BC,CD\}$ $F1=\{A\rightarrow B, B\rightarrow C\}$; $F2=\{B\rightarrow C, C\rightarrow D\}$;

(1)如果 F1 是 R上的函数依赖集，此时 ρ 是无损联接分解吗 ?若不是，试举出反例。

(2)如果 F2 是 R上的函数依赖集呢 ?

(1)不是无损联接。可由算法 4.2 判断或由定理 4.8 判断。

根据算法 4.2

A

B

C

D

AB

a1

a2

b13

b14

BC

b21

a2

a3

b24

CD

b31

b32

a3

a4

A

B

C

D

a1

a2

a3
b14

b21
a2
a
3
b24

b31
b32
a3
a4

- (1) 构造表
(2)根据 $A \rightarrow B, B \rightarrow C$ 进行处理

结果没有出现一行全 a 的情况，所以它不是无损联接。举例如下：
设模式 R 的一关系 r 为{(a1b1c1d1),(a2b2c1d2)}
则有： $r_1 = \pi_{AB}(r) = \{(a1b1), (a2b2)\}$
 $r_2 = \pi_{BC}(r) = \{(b1c1), (b2c1)\}$
 $r_3 = \pi_{CD}(r) = \{(c1d1), (c1d2)\}$
令 $a = r_1 r_2 r_3 = \{(a1b1c1d1), (a1b1c1d2), (a2b2c1d1), (a2b2c1d2)\}$
 $r \neq a$, 所以 ρ 不是无损联接。

- (2)如果 F2 是 R 上的函数依赖，则可以判断， ρ 是无损联接。判断过程同上。

4.11 设关系模式 R(S#,C#,GRADE,TNAME,TADDR),属性分别表示学生学号、选修课程的编号,成绩、任课教师地址等意义。如果规定,每个学生每学一门课只有一个成绩;每门课只有一个教师任教;每个教师只有一个地址(此处不允许教师同名同姓)。

- (1)试写出关系模式 R 基本的函数依赖和候选键。
(2)试把 R 分解成 2NF 模式集并说明理由。
(3)试把 R 分解成 3NF 模式集,并说明理由。

(1) $F = \{(S\#,C\#) \rightarrow GRADE, C\# \rightarrow TNAME, TNAME \rightarrow TADDR\}$
候选键是 (S#,C#)。

(2)在模式 R 中, TNAME 不完全依赖于键 (S#, C#),因此需进行分解, 可分解为下列两个关系。
 $SC = \{S\#,C\#,GRADE\}$ $C = \{C\#,TNAME,TADDR\}$
分解后, SC中, GRADE完全依赖于候选键 (S#,C#),在 C 中,主属性是 C#, TNAME、TADDR均

完全依赖于 C#。因此，该分解符合 2NF 模式。

(3)3NF：若每个关系模式是 2NF，则每个非主属性都不传递于 R 的候选键。

按上述已分好的两个模式，SC 中已满足“每个非主属性都不传递于 R 的候选键”，已是 3NF，而在 C 中，C#→TNAME, TNAME→TADDR, TADDR传递依赖于 C#，因此还需分成两个模式：

CT(C#,TNAME), T(TNAME,TADDR)

分解后，总共有 SC={S#,C#,GRADE},CT(C#,TNAME), T(TNAME,TADDR) 模式。

该分解符合 3NF 模式。

4.12 图 4.6 表示一个公司各部门的层次结构，对每个部门，数据库中包含部门号 (唯一的)D#，预算费 (BUDGET)以及此部门领导人员的职工号 (唯一的)E#等信息。对每一个部门，还存有部门的全部职工，生产科研项目以及办公室的信息。

职工信息包括：职工号，他所参加的生产科研项目号 (J#)，他所在办公室的电话号 (PHONE#)。

生产科研项目包含：项目号 (唯一的)，预算费。

办公室信息包含：办公室号 (唯一的)，面积。

对每个职工，数据库中有他曾担任过的职务以及担任某一职务时的工资历史。对每个办公室包含此办公室中全部电话号吗的信息。

请给出你认为合理的数据依赖，把这个层次结构转换成一组规范化的关系。

提示：此题可分步完成，先转换成一组 1NF 的关系，然后逐步转换成 2NF, 3NF, ...。

先得到一个泛关系的模式如下：

D={D#,Manager_E#,Budget, E#,J#,Phone#,Business,Sa_History,Office#,Area}

D#:部门号, Manager_E#: 部门领导人员的职工号, E#:职工号, J#:生产科研项目号, Phone#:办公室的电话号, Business:职工职务, Sa_History:工资历史, Office#: 办公室号, Area:办公室面积

根据所给信息，给出下列数据依赖：

F={D#→Manager_E#, E#→Office#,(E#,Business)→Sa_History,J#→Budget,E#→J#, Office#→Area,Office→D#,#Phone#→Office#}

(假设一个部门可能有多个办公室，有多个项目，一个办公室只属于一个部门，有多部电话，一个员工只参加一个项目，一个项目可能属于多个部门)

只要保证每个属性值不可分割，以上范式即为 1NF。候选键为 (E#,Business, Phone#)

转换成 2NF 关系 (消除局部依赖)：

Em_Dep(E#,D#,Manager_E#,Office#,Area,J#, Budget)

对应 F={D#→Manager_E#, E#→Office#,J#→Budget,E#→J#, Office#→Area, Office→D#}

History(E#,Business,History)

对应 F={(E#,Business)→Sa_History}

Phone(Phone#,Office#)

对应 F={Phone#→Office#}

转换成 3NF 关系 (消除非主属性对侯选键的传递依赖)：

Department(D#,Manager_E#)

Office(Office#,Area , D#)

Emproee(E#,J#,Office#)

History(E#,Business,History)

Phone(Phone#,Office#)

Project(J#,Budget)

注意：由于对题意理解的不同，可能答案不唯一。

4.13 设关系模式 $R(ABC)$ 上有一个多值依赖 $A \twoheadrightarrow B$ 。如果已知 R 的当前关系中存在三组 (ab1c1)、(ab2c2)和 (ab3c3),那么这个关系中至少还应存在哪些元组 ?

从多值依赖的定义可以得出，至少应存在下列元组：

(ab1c2)、(ab1c3)、(ab2c1)、(ab2c3)、(ab3c1)、(ab3c2)

5.1 名词解释

(1) 数据库设计：数据库设计是指对于一个给定的应用环境，提供一个确定最优数据模型与处理模式的逻辑设计，以及一个确定数据库存储结构与存取方法的物理设计，建立起既能反映现实世界信息和信息联系，满足用户数据要求和加工要求，又能被某个数据库管理系统所接受，同时能实现系统目标，并有效存取数据的数据库。

(2) 数据库工程：指数据库应用系统的开发，它是一项软件工程 ,但有数据库应用自身的特点。

(3) 评审：在数据库设计过程中，评审的目的是为了确认某一阶段的任务是否全部完成，避免重大的疏漏或错误，并在生存期的早期阶段给予纠正，以减少系统研制的成本。

(4) 数据字典：是对系统中数据的详尽描述，它提供对数据库数据描述的集中管理。它的处理功能是存储和检索元数据，并且为数据库管理员提供有关的报告。对数据库设计来说，数据字典是进行详细的数据收集和数据分析所获得的主要成果。数据字典主要包括四个部分：数据项、数据结构、数据流、数据存储。

5.2 数据系统的生存期分成哪几个阶段 ?数据库结构的设计在生存期中的地位如何 ?

分成七个阶段：规划、需求分析、概念设计、逻辑设计、物理设计、实现、运行和维护。

数据库结构的设计在生存期中的地位很重要，数据库结构的设计包括逻辑设计、物理设计，逻辑设计把概念模式转化为与选用的具体机器上的 DBMS 所支持的数据模型相符合的逻辑结构，而物理设计主要是设计 DB 在物理设备上的存储结构与存取方法等。

5.3 什么是数据库设计 ?数据库设计过程的输入和输出有哪些内容 ?

数据库设计是指对于一个给定的应用环境，提供一个确定最优数据模型与处理模式的逻辑设计，以及一个确定数据库存储结构与存取方法的物理设计，建立起既能反映现实世界信息和信息联系，满足用户数据要求和加工要求，又能被某个数据库管理系统所接受，同时能实现系统目标，并有效存取数据的数据库。

数据库设计过程的输入包括四部分内容： 1)总体信息需求； 2)处理需求； (3)DBMS 的特征； (4)硬件和 OS(操作系统)特征。

数据库设计过程的输出包括两部分：

(1)完整的数据库结构，其中包括逻辑结构和物理结构；

(2)基于数据库结构和处理需求的应用程序的设计原则。这些输出一般以说明书的形式出现。

5.4 基于数据库系统生存期的数据库设计分成哪几个阶段？

分为 5 个阶段：简记为 (规需概逻物)：规划、需求分析、概念设计、逻辑设计和物理设计五个阶段。

5.5 什么是比较好的数据库设计方法？数据库设计方法应包括哪些内容？

一个好的数据库设计方法应该能在合理的期限内，以合理的工作量产生一个有实用价值的数据库结构。

一种实用的数据库设计方法应包括以下内容：设计过程、设计技术、评价准则、信息需求、描述机制。

5.6 数据库设计的规划阶段应做哪些事情？

数据库设计中的规划阶段的主要任务是进行建立数据库的必要性及可行性分析，确定数据库系统在组织中和信息系统中的地位，以及各个数据库之间的联系。

5.7 数据库设计的需求分析阶段是如何实现的？目标是什么？

数据库设计的需求分析通过三步来完成：即需求信息的收集、分析整理和评审。

其目的在于对系统的应用情况作全面详细的调查，确定企业组织的目标，收集支持系统总的设计目标的基础数据和对这些数据的要求，确定用户的需求，并把这些要求写成用户和数据设计者都能够接受的文档。

5.8 评审在数据库设计中有什么重要作用？为什么允许设计过程中有多次的回溯与反复？

评审的作用在于确认某一阶段的任务是否全部完成，通过评审可以及早发现系统设计中的错误，并在生存期的早期阶段给予纠正，以减少系统研制的成本。

如果在数据库已经实现时再发现设计中的错误，那么代价比较大。因此应该允许设计过程的回溯与反复。设计过程需要根据评审意见修改所提交的阶段设计成果，有时修改甚至要回溯到前面的某一阶段，进行部分乃至全部重新设计。

5.9 数据字典的内容和作用是什么？

数据字典的内容一般包括：数据项、数据结构、数据流、数据存储和加工过程。

其作用是对系统中数据做出详尽的描述，提供对数据库数据的集中管理。

5.10 对概念模型有些什么要求？

对概念模型一般有以下要求：

(1) 概念模型是对现实世界的抽象和概括，它应真实、充分地反映现实世界中事物和事物之间的联系，具有丰富的语义表达能力，能表达用户的各种需求，包括描述现实世界中各种对象及其复杂联系、用户对数据对象的处理要求和手段。

(2) 概念模型应简洁、明晰，独立于机器、容易理解、方便数据库设计人员与应用人员交换意见，使用户能积极参与数据库的设计工作。

(3) 概念模型应易于变动。当应用环境 and 应用要求改变时，容易对概念模型修改和补充。

(4) 概念模型应很容易向关系、层次或网状等各种数据模型转换, 易于从概念模式导出也 DBMS 有关的逻辑模式。

5.11 概念设计的具体步骤是什么 ?

概念设计的主要步骤为:

- (1) 进行数据抽象、设计局部概念模式;
- (2) 将局部概念模式综合成全局概念模式;
- (3) 评审。

(具体解释参见教材 p131)

5.12 什么是数据抽象 ?主要有哪两种形式的抽象 ?数据抽象在数据库设计过程中起什么作用?

数据抽象是对人、物、事或概念的人为处理, 它抽取人们关心的共同特性, 忽略非本质的细节, 并把这些特性用各种概念精确地加以描述, 这些概念组成了某种模型。

数据抽象有两种形式:

- (1) 系统状态的抽象, 即抽象对象;
- (2) 系统转换的抽象, 即抽象运算。

数据抽象是概念设计中非常重要的一步。通过数据抽象, 可以将现实世界中的客观对象首先抽象为不依赖任何具体机器的信息结构。

5.13 什么是 ER图?构成 ER图的基本要素是什么 ?

ER图是用来表示数据库概念设计 ER模型的工具, 它提供了表示实体、属性和联系的方法。构成 ER图的基本要素是实体、属性和联系。

5.14 试述采用 ER方法的数据库概念设计的过程。

利用 ER方法进行数据库的概念设计, 可分成三步进行: 首先设计局部 ER模式, 然后把各局部 ER模式综合成一个全局 ER模式, 最后对全局 ER模式进行优化, 得到最终的 ER模式, 即概念模式。

5.15 逻辑设计的目的是什么 ?试述逻辑设计过程的输入和输出环境。

逻辑设计的目的是把概念设计阶段设计好的基本 ER图转换为与选用的具体机器上的 DBMS 所支持的数据模型相符合的逻辑结构 (包括数据库模式和外模式)。

逻辑设计过程中的输入信息有:

- (1) 独立于 DBMS 的概念模式, 即概念设计阶段产生的所有局部和全局概念模式;
- (2) 处理需求, 即需求分析阶段产生的业务活动分析结果;
- (3) 约束条件, 即完整性、一致性、安全性要求及响应时间要求等;
- (4) DBMS 特性, 即特定的 DBMS 特性, 即特定的 DBMS 所支持的模式、子模式和程序语法的形式规则。

逻辑设计过程输出的信息有:

- (1) DBMS 可处理的模式;
- (2) 子模式;

(3)应用程序设计指南；

(4)物理设计指南。

5.16 试述逻辑设计阶段的主要内容。

逻辑设计主要是把概念模式转换成 DBMS 能处理的模式。转换过程中要对模式进行评价和性能测试，以便获得较好的模式设计。

逻辑设计的主要步骤包括：

- (1)初始模式的形成
- (2)子模式设计
- (3)应用程序设计梗概
- (4)模式评价
- (5)修正模式。

5.17 规范化理论对数据库设计有什么指导意义？

在概念设计阶段，已经把关系规范化的某些思想用作构造实体类型和联系类型的标准，在逻辑设计阶段，仍然要使用关系规范化的理论来设计模式和评价模式。规范化的目的是减少乃至消除关系模式中存在的各种异常，改善完整性，一致性和存储效率。

5.18 什么是数据库结构的物理设计？试述其具体步骤。

数据库结构的物理设计是指对一个给定的逻辑数据模型选取一个最适合应用环境的物理结构的过程，所谓数据库的物理结构主要指数据库在物理设备上的存储结构和存取方法。

物理设计的步骤为：

- (1)设计存储记录结构，包括记录的组成、数据项的类型和长度，以及逻辑记录到存储记录的映射；
- (2)确定数据存储安排；
- (3)设计访问方法，为存储在物理设备上的数据提供存储和检索的能力；
- (4)进行完整性和安全性的分析、设计；
- (5)程序设计。

5.19 数据实现阶段主要做哪几件事情？

数据库实现阶段的主要工作有以下几点：

- (1) 建立实际数据库结构
- (2) 试运行
- (3) 装入数据

(具体解释参见教材 p150)

5.20 什么是数据的再组织设计？试述其重要性。

对数据的概念模式、逻辑结构或物理结构的改变称作数据再组织。

数据再组织，通常是由于环境，需求的变化或性能原因而进行的，如信息定义的改变，增加新的数据类型，对原有的数据提出了新的使用要求，改用具有不同物理特征的新存储设备以

及数据库性能下降等都要求进行数据库的重新组织。

5.21 数据系统投入运行后，有哪些维护工作？

数据系统投入运行后，主要维护工作有：

- (1)维护数据库的安全性与完整性控制及系统的转储和恢复；
- (2)性能的监督、分析与改进；
- (3)增加新功能；
- (4)发现错误，修改错误。

6.1 名词解释

- (1)事务：事务是指一个操作序列，这些操作要么什么都做，要么都不做，是一个不可分割的工作单位，是数据库环境中的逻辑工作单位，相当于操作系统环境下的 "进程" 概念。
- (2)封锁：封锁就是事务可以向系统发出请求，对某个数据对象加锁，此事务对这个数据对象有一定控制，而其他事务则不能更新数据直到事务释放它的锁为止。
- (3)X 封锁：如果事务 T 对数据 R 实现 X 封锁，那么其他的事务要等 T 解除 X 封锁以后，才能对这个数据进行封锁。只有获准 X 封锁的事务，才能对被封锁的数据进行修改。
- (4)PX 协议：X 封锁的规则称为 PX 协议，其内容为：任何企图更新记录 R 的事务必须先执行 LOCK X(R) 操作，以获得对该记录进行寻址的能力，并对它取得 X 封锁。如果未获得 X 封锁，那么这个事务进入等待状态，一直到获准 X 封锁，事务继续进行。
- (5)PXC 协议：它由 PX 协议及一条规则 "X 封锁必须保留到事务终点 (COMMIT 或 ROLLBACK)" 组成。
- (6)死锁：有两个或以上的事务处于等待状态，每个事务都在等待另一个事务解除封锁，它才能继续执行下去，结果任何一个事务都无法执行，这种现象就是死锁。
- (7)串行调度：多个事务依次执行，称为串行调度。
- (8)并发调度：利用分时的方法，同时处理多个事务，称为事务的并发调度。
- (9)可串行化调度：如果某事务集的一个并发调度的结果与某个串行调度等价，则称此并发调度是可串行化调度。
- (10)不可串行化调度：某事务集的一个并发调度结果，如果与任一串行调度均不等价，则该并发调度是不可串行化调度。
- (11)S 封锁：共享型封锁。果事务 T 对某数据 R 加上 S 封锁，那么其它事务对数据 R 的 X 封锁便不能成功，而对数据 R 的 S 封锁请求可以成功。这就保证了其他事务可以读取 R 但不能修改 R，直到事务 T 释放 S 封锁。
- (12)PS 协议：任何要更新记录 R 的事务必须先执行 LOCK S(R) 操作，以获得对该记录寻址的能力并对它取得 S 封锁。如果未获准 S 封锁，那么这个事务进入等待状态，一直到获准 S 封锁，事务才继续进行下去。当事务获准对记录 R 的 S 封锁后，在记录 R 修改前必须把 S 封锁升级为 X 封锁。
- (13)PSC 协议：由 PS 协议和 "将 S 封锁保持到事务终点" 组成。任何更新记录 R 的事务必须先执行 LOCK S(R) 操作，以获得对该记录寻址的能力并对它取得 S 封锁。如果未获准 S 封锁，那么这个事务进入等待状态，一直到获准 S 封锁，事务才继续进行下去。并将 S 封锁保持到事务终点。

(14)两段封锁协议：（1）在对任何数据进行读写操作之前，事务首先要获得对该数据的封锁；
（2）在释放一个封锁之后，事务不再获得任何其他封锁。

6.2 试叙事务的四个性，并解释每一个性质对 DBS有什么益处？
事务的四个性是：原子性、一致性、隔离性和持久性。

- (1)原子性（具体含义见教材）：事务中所有操作应视为整体，不可分割。
- (2)一致性（具体含义见教材）：一个事务独立执行的结果将保证数据库的一致性，即数据不会因事务的执行而遭受破坏。
- (3)隔离性（具体含义见教材）：隔离性要求在并发事务被执行时，系统应保证与这些事务先后单独执行时结果一样，使事务如同在单用户环境下执行一样。
- (4)持久性（具体含义见教材）：要求事务对数据库的所有更新应永久地反映在数据库中。

6.3 事务的 COMMIT 操作和 ROLLBACK操作各做些什么事情？
COMMIT 操作表示事务成功地结束（提交），此时告诉系统，数据库要进入一个新的正确状态，该事务对数据库的所有更新都已交付实施。
ROLLBACK操作表示事务不成功地结束，此时告诉系统，已发生错误，数据库可能处在不正确的状态，该事务对数据库的更新必须被撤销，数据库应恢复该事务到初始状态。

6.4 UNDO 操作和 REDO操作各做些什么事情？
UNDO 操作是反向扫描“日志”文件，撤销对数据库的更新操作，使数据库恢复到更新前的状态；
REDO操作正向扫描日志文件，重新做一次更新，使数据库恢复到更新后的状态。

6.5 DBS中有哪些类型的故障？哪些故障破坏了数据库？哪些故障未破坏数据库，但其中某些数据变得不正确？
数据库系统故障有事务故障、系统故障、介质故障。
其中介质故障破坏数据库，事务故障、系统故障未破坏数据库但使其中某些数据变得不正确。

6.6 什么是“运行记录优先原则”？其作用是什么？
在数据库系统中，写一个修改到数据库中和写一个表示这个修改的登记记录到日志文件中是两个不同的操作，在这两个操作之间有可能发生故障。这时，如果先写了数据库修改，而在运行记录中没有登记这个修改，则以后就无法恢复这个修改，那么以后也就无法撤消这个修改。为了安全起见，采用“运行记录优先原则”。它包括两点：
（1）至少要等相应运行记录已经写入“日志”文件后，才能允许事务往数据库中写记录。
（2）直至事务的所有运行记录都已写入运行“日志”文件后，才能允许事务完成“END TRANSACTION”处理。

采用“运行记录优先原则”，可以保证所做的操作都记录在日志中，便于故障处理，维护数据库的一致性。

6.7 什么是数据库的恢复？恢复的基本原则是什么，恢复如何实现？
数据库的恢复是指当数据库系统发生故障时，通过一些技术，使数据库从被破坏、不正确的状态恢复到最近一个正确的状态。
恢复的基本原则就是冗余，即数据的重复存储。恢复的常用方法有：（1）定期对整个数据库

进行复制或转储；（2）建立日志文件；（3）恢复。用 REDO或 UNDO 处理。

6.8 数据库的并发操作会带来哪些问题？如何解决？

数据库的并发操作会带来三类问题：

- （1）丢失更新问题；（具体内容见教材 P158）
- （2）不一致分析问题
- （3）“脏数据”的读出。

解决的办法通常是采用“封锁”技术。

6.9 有哪些“丢失更新”问题？如何处理？

（1）并发调度引起的“丢失更新”问题。在并发调度中，如两个事务同时对某数据库进行更新操作，在操作中丢失了其中某个事务对某数据的更新操作。处理的办法就是对并发操作采用 X 封锁技术。

（2）由恢复引起的更新丢失现象。采用 X 封锁以后，先执行事务 T1，再执行事务 T2，此时，如果 T2 做 ROLLBACK 操作，就把事务 T2 的更新丢失了。处理方式是采用 PXC 协议，不允许事务 T1 执行解锁操作直到事务执行到终点（COMMIT 或 ROLLBACK）。

6.10 为什么 DML 只提供解除 S 封锁的操作，而不提供解除 X 封锁的操作？

为了防止由恢复引起的更新丢失现象，PXC 协议规定：X 封锁必须保留到事务终点（COMMIT 或 ROLLBACK）。因此在 DML 中，COMMIT 或 ROLLBACK 的语义中包含了 X 封锁的解除，不用提供解除 X 封锁的操作。

而对于 S 封锁而言，它是一种共享型的封锁（读取但不能修改），其数据不可能是未提交的修改，封锁不必等提交后才能解除，因此可以由事务用 UNLOCK 解除各自对数据的封锁。

6.11 为什么有些封锁需保留到事务终点，而有些封锁可随时解除？

因为有的封锁需要更新数据，保留到终点才能确认是否提交或回退以避免脏数据的读出，所以这些封锁需要保留到事务终点。

而有些事务全部或部分操作只是读取数据，不涉及更新数据操作，因此这一部分操作的封锁可以随时解除。

6.12 死锁的发生是坏事还是好事？试说明理由。如何解除死锁状态？

死锁的发生即是坏事又是好事。

- （1）使并发事务不能继续执行下去，造成时间开销却不产生结果。
- （2）在某些时候我们要利用它来解决更新操作导致的数据库不一致状态。

解除死锁状态：抽出某个事务作为牺牲品，把它撤消，做回退操作，解除它的所有封锁，并恢复到初始状态。

6.13 试叙述“串行调度”与“可串行化调度”的区别。

串行调度是多个事务按照一定的次序依次执行；在某一时刻只有一个事务在执行。

而可串行化调度是并发调度中的一个概念。在并发调度中，某一时刻有多个事务同时被处理。如果某事务集的一个并发调度的结果与某个串行调度等价，则称此并发调度是可串行化调度。

（以上几题具体可参照书上 p20-21）

6.14 什么是数据库的完整性？DBMS的完整性子系统的功能是什么？

数据库的完整性是指数据的正确性和相容性。DBMS完整性子系统的功能是：

- (1)监督事务的执行，并测试是否违反完整性规则；
- (2)如有违反，则采取恰当的操作，如拒绝、报告违反情况，改正错误等方法进行处理。

6.15 完整性规则由哪几个部分组成？关系数据库的完整性规则有哪几类？

完整性规则由三部分组成：

触发条件：即什么时候使用规则进行检查；

约束条件：即要检查什么样的错误；

ELSE子句：即查出错误后该如何处理。

完整性规则有以下三类：

域完整性规则，用于定义属性的取值范围；

域联系的规则，定义一个或多个关系中，属性值间的联系、影响和约束。

关系完整性规则，定义更新操作对数据库中值的影响和限制。

6.16 试详述 SQL中的完整性约束机制？

SQL中的完整性约束规则有主键约束、外键约束、属性值约束和全局约束等多种形式。

△主键约束。它是数据中最重要的一种约束。在关系中主键值不允许空，也不允许出现重复，体现了关系要满足实体完整性规则。主键可用主键子句或主键短语进行定义。

△外键约束。根据参照完整性规则，依赖关系中外键或者为空值，或者是基本关系（参照关系）中的该键的某个值。外键用外键关系子句定义，并考虑删除基本关系元组或修改基本关系的主键值的影响，依赖关系可按需要采用 RESTRICT SET NULL CASCADE方式。

△属性值约束。当要求某个属性的值不允许空值时，那么可以在属性定义后加上关键字：

NOT NULL，这是非空值约束。还可以用 CHECK子句对一个属性值加以限制以及使用域约束子句 CREATE DOMAIN 定义新域并加以属性值检查。

△全局约束。在关系定义时，可以说明一些比较复杂的完整性约束，这些约束涉及到多个属性间的联系或不同关系间的联系，称为全局约束。主要有基于元组的检查子句和断言。前者是对单个关系的元组值加以约束，后者则可对多个关系或聚合操作有关的完整性约束进行定义。

6.17 参照完整性规则在 SQL可以用哪几种方式实现？删除基本关系的元组时，依赖关系可以采取的做法有哪三种？修改基本关系的主键值时，依赖关系可以采取的做法有哪三种？

参照完整性规则要求“不引用不存在的实体”，参照完整性规则在 SQL可用以下几种方式实现：

- (1)在 SQL中采用外键子句定义外键，并考虑删除基本关系元组或修改基本关系的主键值，对依赖关系产生的影响；
- (2)在属性值上进行约束如基于属性的检查；
- (3)全局约束中的基于元组的检查子句等。

删除基本关系元组或修改基本关系的主键值时，依赖关系可以采用的做法有：

△RESTRICT方式：只有当依赖关系中没有一个外键值与基本关系中要删除 /修改的主键值相对应时，系统才能执行删除 /修改操作，否则拒绝删除或修改。

△SET NULL方式：删除基本元组时， 将依赖关系中所有与基本关系中被删除主键值相对应的外键值置为空值。 修改基本关系的主键值时， 将依赖关系中所有与基本关系中被修改主键值相对应的外键值置为空值。

△CASCADE方式：若删除则将依赖关系中所有外键值与基本关系中要删除的主键值相对应的元组一并删除， 若修改则将依赖关系中所有与基本关系中要修改的主键值相对应的外键值一并修改为新值。

6.18 试对 SQL2中的基于属性的检查约束、 基于元组的检查约束和断言三种完整性约束进行比较：各说明什么对象 何时激活 能保证数据库的一致性吗 ？

约束形式

说明对象

激活条件

是否保证一致性

基于属性的检查

只对一个属性值加以约束

插入或修改属性值时

不一定

基于元组的检查

对单个关系的元组值加以约束

在插入或修改元组时

不一定

断言

多个关系或聚合操作

任何变动

保证

6.19 设教学数据库的模式如下：

S(S#,SNAME,AGE,SEX)

SC(S#,C#,GRADE)

C(C#,CNAME,TEACHER)

试用多种方式定义下列完整性约束：

(1)在关系 S中插入学生年龄值应在 16～25 岁之间

(2)在关系 SC中插入元组时，其 S#值和 C#值必须分别在 S和 C中出现。

(3)在关系 SC中修改 GRADE值时，必须仍在 0～100 之间。

(4)在删除关系 C中一个元组时，首先要把关系 SC中具有同样 C#的元组全部删去。

(5)在关系 S中把某个 S#值修改为新值时， 必须同时把关系 SC中那些同样的 S#值也修改为新

值。

(1)定义 S时采用检查子句：

```
CREAT TABLE S(  
S# CHAR(4),  
SNAME char (10) NOT NULL ,  
AGE SMALLINT ,  
PRIMARY key(S#),  
CHECK (AGE>=16 and AGE<=25) )
```

(2)采用外键子句约束

```
CREAT TABLE SC(  
S# CHAR(4),  
C# CHAR(4),  
GRADE SMALLINT,  
FOREIGN key(S#) REFERENCE S(S#),  
FOREIGN key(C#) REFERENCE C(C#)  
)
```

(3)采用元组检查

```
CREAT TABLE SC(  
S# CHAR(4),  
C# CHAR(4),  
GRADE SMALLINT,  
FOREIGN key(S#) REFERENCE S(S#),  
FOREIGN key(C#) REFERENCE C(C#),  
CHECK (GRADE>=0 and AGE<=100) )
```

FOREIGN key(C#) RE

(4)采用外键约束

```
CREAT TABLE SC(  
S# CHAR(4),  
C# CHAR(4),  
GRADE SMALLINT,  
FOREIGN key(S#) REFERENCE S(S#),  
FOREIGN key(C#) REFERENCE C(C#)  
)
```

若改为：在删除关系 C 中一个元组时，同时把关系 SC中具有同样 C#的元组全部删去，则为：

```
.....  
FOREIGN key(C#) REFERENCE C(C#) ON DELETE CASCADE  
.....
```

(5)采用外键约束

```
CREAT TABLE SC(  
  S# CHAR(4),  
  C# CHAR(4),  
  GRADE SMALLINT,  
  FOREIGN key(S#) REFERENCE S(S#) ON UPDATE CASCADE ,  
  FOREIGN key(C#) REFERENCE C(C#)  
)
```

6.20 在教学数据库的关系 S、SC、C 中，试用 SQL2 的断言机制定义下列两个完整性约束： (1)
学生必须在选修 Maths 课后，才能选修其他课程。 (2)每个男学生最多选修 20 门课程

(1)

```
CREAT ASSERTION ASSE1 CHECK  
( NOT EXISTS  
  ( SELECT S FROM SC  
    WHERE C# IN  
      (SELECT C#  
        FROM C  
        WHERE CNAME<>'MATHS')  
    AND S# NOT IN  
      (SELECT S# FROM SC  
        WHERE C# IN  
          (SELECT C#  
            FROM C  
            WHERE CNAME='MATHS'))));
```

(2)

```
CREAT ASSERTION ASSE2 CHECK  
( ALL(SELECT COUNT (SC.C#)  
      FROM S,SC  
      WHERE S.S#=SC.S AND SEX='M'  
      GROUP BY S#)<=20);
```

6.21 什么是数据库的安全性？有哪些安全措施？

数据库的安全性是指保护数据库防止不合法的使用，以免数据的泄漏、非法更改和破坏。

可以从以下方面设置数据库的安全措施：环境级、职员级、OS 级、网络级和数据库系统级。

(具体可参见教材 P173)

6.22 对银行的数据库系统应采取哪些安全措施？分别属于哪一级？

答案基本同上一题。

6.23 什么是“权限”？用户访问数据库可以有哪些权限？对数据库模式有哪些修改权限？

用户使用数据库的方式称为权限。

用户访问数据库的权限有：读权限、插入权限、修改权限、删除权限。

用户修改数据库模式的权限有索引权限、资源权限、修改权限、撤销权限。

6.24 试解释权限的转授与回收。

在数据库系统中,为了保证数据的安全性,用户对数据的操作必须首先从 DBA 处获得权限,才能进行对数据的操作。同时数据库系统也允许用户将获得的权限转授给其他用户,也允许把已授给其他用户的权限再回收上来。但应保证转授出去的权限能收得回来。

6.25 SQL 语言中的视图机制有哪些优点 ?

视图机制使系统具有三个优点:数据安全性、数据独立性和操作简便性。

6.26 SQL2 中用户权限有哪几类 ?并作必要的解释。

SQL2 中定义了六类用户权限,分别是:

(1)SELECT 允许用户对关系或视图执行 SELECT 操作。

(2)INSERT: 允许用户对关系或视图执行 INSERT 操作,如果还说明了一个属性表,那么表示只能插入这些属性的值,关系中其他属性值将置为缺省值或为空值。

(3)DELETE 允许用户对关系或视图执行 DELETE 操作。

(4)UPDATE: 允许用户对关系或视图执行 UPDATE 操作,如果还说明了一个属性表,那么表示只能修改这些属性的值,否则表示可以修改关系中任一属性值。

(5)REFERENCES 允许用户定义新关系时,引用其他关系的主键作为外键。

(6)USAGE: 允许用户使用定义的域。

6.27 数据加密法有什么优点 ?如何实现 ?

数据加密可以更好地保证数据的完全性。加密采用一定的加密算法把源文变为密文来实现。常用的加密算法有 "替换方法"和明键加密法。

7.1 名词解释

(1) 集中式 DBS: 是指数据库中的数据集中存储在一台计算机上,数据的处理集中在一台计算机上完成。

(2) 分布式 DBS: 是指数据存放在计算机网络的不同场地的计算机中,每一场地都有自治处理能力并完成局部应用;而每一场地也参与 (至少一种)全局应用程序的执行,全局应用程序可通过网络通信访问系统中的多个场地的数据。

(3) 分布式 DB: 分布式数据库,是指计算机网络环境中各场地上数据库的逻辑集合。

(4) 分布式 DBMS: 分布式数据库管理系统。是指分布式数据库系统中的一组软件,它负责管理分布环境下逻辑集成数据的存取、一致性、有效性和完备性。同时,由于数据的分布性,在管理机制还必须具有计算机网络通信协议上的分布管理特性。

(5) 局部自治性: 是指有独立处理能力并能完成的局部应用。

(6) 数据分配 (数据分布): 是指数据计算机网络各场地上的分配策略。

(7) 数据复制: 是指数据在每个场地重复存储。

(8) 数据分片: 是指分布式数据库中的数据可以被分割和复制在网络场地的各个物理数据库中。一般数据存放的单位不是关系而是片段,一个片段是关系的一部分。数据分片是通过关系代数的基本运算实现的。

(9) 水平分片: 是指按一定条件把全局关系的所有元组划分成若干不相交的子集,每个子集为关系的一个片段。

(10) 垂直分片: 把一个全局关系的属性集分成若干子集,并在这些子集上做投影运算,每个投影为垂直分片。

(11) 分布透明性：指用户不必关心数据的逻辑分片，不必关心数据物理位置的细节，也不必关心各个场地上的数据库的数据模型。

(12) 分片透明性：分片透明性是最高层次的分布透明性，是指用户或应用程序只对全局关系进行操作而不必考虑数据的分片。

(13) 位置透明性：位置透明性位于分片视图与分配视图之间。是指用户或应用程序应当了解分片情况，但不必了解片段的存储场地。

(14) 局部数据模型透明性：这个透明性位于分配视图与局部概念视图之间，指用户或应用程序要了解分片及各片段存储的场地，但不必了解局部场地上使用的是何种数据模型。

(15) 复制透明性：即用户不必关心数据库在网络中各个结点的数据库复制情况，更新操作引起的波及由系统去处理。

7.2 试叙数据从集中存储、分散存储到分布存储的演变过程。

早期的数据库系统都是属于集中式数据库系统，数据集中存储在一台计算机上。该方式有一定优点，比如，价格合理，易于管理，数据冗余少等等。但是，随着数据库应用的不断发展，数据库规模不断扩大，如采用集中存储的数据库，其系统的设计和操作都比较复杂，系统显得不灵活而且安全性也较差。因此采用了数据分散存储的办法，将数据库分成多个，建立多台计算机上，此时的各个数据库的管理和应用程序均是分开并独立的。它们之间不存在数据通信联系。随着计算机网络通信的发展，有可能把分散在各处的数据库系统通过网络通信连接起来，这样形成的系统称为分布式的数据库系统，这时的数据库虽然分别存在各地的计算机上，但它们之间通过分布式数据管理系统可以进行联系。

7.3 与集中式 DBS、分散式 DBS相比，分布式 DBS有哪些特点？

与集中式 DBS相比，分布式 DBS中数据库的存储是分布性的，即数据不是存储在同一场地，而是分布存储在多个场地。与分散式 DBS相比，分布式 DBS中数据库具有“逻辑整体性”，即表面上看，分布式数据的数据库分散在各个场地，但这些数据在逻辑上却是一个整体，如同一个集中式数据库。

7.4 试解释下列术语：同构同质型 DDBS、同构异质型 DDBS、异构 DDBS

(1) 同构同质型 DDBS：是指各个场地都采用同一类型的数据模型，并且是同一型号数据库管理系统。

(2) 同构异质型 DDBS：是指各个场地都采用同一类型的数据模型，但是数据库管理系统是不同型号的。

(3) 异构型 DDBS：是指各个场地的数据模型是不同的类型。

7.5 分布式数据库系统有哪些优点和缺点？

分布式数据库系统的优点：

- (1) 具有灵活的体系结构
- (2) 适应分布式的管理和控制机构
- (3) 经济性能优越
- (4) 系统的可靠性高、可用性好
- (5) 局部应用的响应速度快
- (6) 可扩展性好，易于集成现有系统

分布式数据库系统的缺点：

- (1) 系统开销大，主要花在通信部分；
- (2) 复杂的存取结构，原来在集中式系统中有效存取数据的技术，在分布式系统中都不再适用；
- (3) 数据的安全性和保密性较难处理。

7.6 分布式数据库中，“数据分配”有哪些策略？“数据分片”有哪些策略？数据分片的准则是什么？

数据分配是指数据在计算机网络各场地上的分配策略。包括：

- (1) 集中式：所有数据均安排在同一个场地上。
- (2) 分割式：所有数据只有一份，分别被安置在若干个场地。
- (3) 全复制式：数据在每个场地重复存储。
- (4) 混合式：数据库分成若干可相交的子集，每一子集安置在一个或多个场地上，但是每一场地未必保存全部数据。

数据分片的方式有以下三种：

- (1) 水平分片：按一定的条件把全局关系的所有元组划分成若干不相交的子集，每个子集为关系的一个片段。
- (2) 垂直分片：把一个全局关系的属性集分成若干子集，并在这些子集上做投影运算，每个投影为垂直分片。
- (3) 混合型分片：将水平分片与垂直分片方式综合使用则为混合型分片。

数据分片应遵循的准则为：

- (1) 完备性条件：必须把全局关系的所有数据映射到各个片段中，绝不允许发生属于全局关系的某个数据不属于任何一个片段。
- (2) 重构条件：划分所采用的方法必须确保能够由各个片段重建全局关系。
- (3) 不相交条件：要求一个全局关系被划分后得到的各个数据片段互相不重叠。

7.7 试叙述分布式 DBS 的体系结构，它有什么特征？

分布式 DBS 的体系结构分为四级：全局外模式、全局概念模式、分片模式和分配模式。

- (1) 全局外模式：它们是全局应用的用户视图，是全局概念模式的子集。
- (2) 全局概念模式：全局概念模式定义了分布式数据库中所有数据的逻辑结构。
- (3) 分片模式：分片模式定义片段以及定义全局关系与片段之间的映象。这种映象是一对多的，即每个片段来自一个全局关系，而一个全局关系可分成多个片段。
- (4) 分配模式：片段是全局关系的逻辑部分，一个片段在物理上可以分配到网络的不同场地上。分配模式根据数据分配策略的选择定义片段的存放场地。

分布式 DBS 的分层体系结构有三个特征：

- (1) 数据分片和数据分配概念的分离，形成了“数据分布独立性”概念。
- (2) 数据冗余的显式控制。
- (3) 局部 DBMS 的独立性。

7.8 试叙述分布式 DBMS 的功能及组成。

主要功能有：

- (1) 接受用户请求，并判定把它送到哪里，或必须访问哪些计算机才能满足该请求。
- (2) 访问网络数据字典，或者至少了解如何请求和使用其中的信息。
- (3) 如果目标数据存储在系统的多个计算机上，就必须进行分布式处理。
- (4) 通信接口功能，在用户、局部 DBMS 和其他计算机的 DBMS 之间进行协调。
- (5) 在一个异构型分布式处理环境中，还需提供数据和进程移植的支持。这里的异构型是指各个场地的硬件、软件之间存在一定差别。

DDBMS 组成——应包括以下四个子系统：

- (1) 查询子系统
- (2) 完整性子系统
- (3) 调度子系统
- (4) 可靠性子系统

7.9 分布式系统中影响查询的主要因素是什么？

分布式系统中，影响查询的主要因素数据经网络的传输时间，数据传输量是衡量查询时间的一个主要指标，不同场地间的联接操作和并操作是影响数据传输量的主要原因。

7.10 基于半联接的查询优化策略的基本思想是什么？

数据在网络中传输时，如果以整个关系传输，则传输数据量大，是一种冗余的做法，而实际上，在联接时并非每个数据都参与操作或有用。因此，考虑到不参与操作的值或无用的值不必经网络来回传输，可以减少传输的数据量。